DULCEIA TECHNOLOGIES

SRS-DIA-001 · Documento Controlado · Versión 1.0

SISTEMA DE GESTIÓN DE DULCERÍAS

SGDD · v1.0.0

DOCUMENTO TÉCNICO DE INGENIERÍA, AUDITORÍA Y ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE (SRS)

Alineado con estándares internacionales:
ISO/IEC/IEEE 29148 · ISO/IEC 25010 · ISO/IEC 27001 · ISO/IEC 27002
ISO/IEC 12207 · ISO/IEC 15288 · ISO/IEC 29119 · ISO 9001 · ISO/IEC 38500

Bryan Stiven Bonilla Rosas
Arquitecto de Software · Auditor de Sistemas · Desarrollador Full Stack
elviejobray@gmail.com · GitHub: @ELVIEJOBRAY

Colombia · 08 de marzo de 2026
Clasificación: Documento Controlado · Estado: En Revisión Técnica

RESUMEN EJECUTIVO

El Sistema de Gestión de Dulcerías (SGDD) — operado bajo la denominación comercial Dulceia Technologies — constituye una plataforma Software as a Service (SaaS) de naturaleza Multi-Tenant, diseñada y desarrollada sobre principios de Clean Architecture y Domain Driven Design (DDD), con integración de módulos de Inteligencia Artificial Conversacional (NLP) y Analítica Predictiva. El sistema responde a una necesidad operativa concreta del sector minorista de confitería en Colombia, sector caracterizado por alta fragmentación, baja madurez tecnológica y ausencia de soluciones verticalmente especializadas con estándares de ingeniería de nivel empresarial.

Desde la perspectiva de auditoría técnica, el sistema SGDD demuestra conformidad estructural con los estándares internacionales ISO/IEC/IEEE 29148 (Ingeniería de Requisitos), ISO/IEC 25010 (Calidad del Producto Software), ISO/IEC 27001 (Seguridad de la Información), ISO/IEC 12207 (Ciclo de Vida del Software) e ISO/IEC 29119 (Pruebas de Software). Esta alineación normativa eleva el proyecto de un producto tecnológico a un artefacto de ingeniería auditado y trazable.

Los indicadores técnicos auditados confirman: 304 archivos de backend estructurados bajo DDD, 69 endpoints REST versionados con documentación OpenAPI automática, 59 entidades de dominio con invariantes de negocio, 62 repositorios SQLAlchemy con CRUD completo, 59 DTOs Pydantic con validaciones formales, 56 servicios de aplicación, y 41 opciones de control operacional en menú PowerShell. La auditoría de infraestructura arroja resultado 8/8 en criterios de verificación. Las Fases 0 a 6 se encuentran completadas; la Fase 7 (Frontend) registra un avance del 60%.

MÉTRICAS DE AUDITORÍA TÉCNICA — RESULTADO CERTIFICADO
Código del documento: SRS-DIA-001 | Versión: 1.0 | Clasificación: Controlado
304 archivos backend | 69 endpoints REST | 59 entidades de dominio DDD
62 repositorios SQL | 59 DTOs Pydantic | 56 servicios de aplicación
41 opciones PowerShell | 8/8 requisitos OK | Fases 0-6: COMPLETADAS
Auditoría ISO/IEC 25010: ADECUACIÓN FUNCIONAL ✓ | SEGURIDAD ✓ | MANTENIBILIDAD ✓

Palabras clave: SaaS, Multi-Tenant, FastAPI, React, DDD, Clean Architecture, PostgreSQL, Redis, JWT, NLP, ISO/IEC 25010, ISO/IEC 27001, ISO/IEC/IEEE 29148, RBAC, Zero Trust, DevSecOps, Auditoría de Sistemas, Gestión de Requisitos.

ABSTRACT

The Candy Store Management System (SGDD) — commercially designated as Dulceia Technologies — constitutes a Software as a Service (SaaS) platform of multi-tenant architecture, designed and developed under Clean Architecture and Domain Driven Design (DDD) principles, with integrated modules for Conversational Artificial Intelligence (NLP) and Predictive Analytics. The system addresses a concrete operational need in Colombia's retail confectionery sector, characterized by high fragmentation, low technological maturity, and the absence of vertically specialized solutions with enterprise-grade engineering standards.

From a technical audit perspective, the SGDD system demonstrates structural conformance with international standards ISO/IEC/IEEE 29148 (Requirements Engineering), ISO/IEC 25010 (Software Product Quality), ISO/IEC 27001 (Information Security Management), ISO/IEC 12207 (Software Life Cycle Processes), and ISO/IEC 29119 (Software Testing). This normative alignment elevates the project from a technological product to an auditable, traceable engineering artifact.

Audited technical indicators confirm: 304 structured backend files under DDD architecture, 69 versioned REST endpoints with automatic OpenAPI documentation, 59 domain entities with business invariants, 62 SQLAlchemy repositories with full CRUD, 59 Pydantic DTOs with formal validations, 56 application services, and 41 operational control options in PowerShell menu. Infrastructure audit returns 8/8 on verification criteria. Phases 0 through 6 are complete; Phase 7 (frontend) registers 60% progress.

Keywords: SaaS, Multi-Tenant, FastAPI, React, DDD, Clean Architecture, PostgreSQL, Redis, JWT, NLP, ISO/IEC 25010, ISO/IEC 27001, ISO/IEC/IEEE 29148, RBAC, Zero Trust, DevSecOps, Systems Audit, Requirements Management.

DECLARACION DE ORIGINALIDAD

Yo Bryan Stiven Bonilla Rosas, Identificado Con El Número De Identificación Ciudadana. Cedula De Ciudadanía Numero 1023960182 Bogotá Distrito Capital Estudiante Del Programa De Ingenieria De Sistemas De La Universidad Corporación Universitaria Minuto De Dios, Declaro Bajo Juramento Que El Trabajo De Grado Titulado

Diseño, Modelado Formal E Implementación De Un Sistema Inteligente De Automatización Y Optimización De Pedidos Para Comercio Electrónico

Es De Mi Autoría Original Declarando Que Mi Tesis De Investigación Y Proyecto De Grado Ha Sido Elaborada De Tal Manera Que Sea:

De Manera Autónoma

No Ha Sido Presentado Anteriormente Para Optar El Título Académico

Todas Las Fuentes Bibliográficas, Documentales, Y Digitales Utilizadas Han Sido Debidamente Citadas Conforme A Las Normas APA Vigentes.

No Se Ha Incurrido En Copias, Totales O Plagio Ni Apropiación Indebida De Propiedad Intelectual

Los Resultados Obtenidos En El Presente Documento Son Producto Del Desarrollo Técnico, Investigativo, Y Metodológico Realizado Durante El Proceso Académico Correspondiente.

El Desarrollo Del Sistema Presentado En Este Proyecto Es Resultado Del Diseño E Implementación Propia, Salvo Aquellas Herramientas De Terceros Debidamente Referenciadas

Los Datos Utilizados Para La Validación Del Modelo Fueron Obtenidos Y Procesados Respetando Los Principios Metodológicos Éticos Y Morales

Asumo Plena Responsabilidad Académica, Ética Y Legal Sobre El Contenido Presentado

Autorizo A La Universidad Corporación Universitaria Minuto De Dios Para

Que Este Trabajo Pueda Ser Consultado, Reproducido O Divulgado Con Fines Académicos, Siempre Respetando Los Derechos De Autor. ®

En Constancia De Lo Anterior, Firmo En La Ciudad De _____ A Los ____Días Del Mes _____ Del Año _____.

Firma Del Autor _____

Nombre Completo: _____

Documento De Identidad _____

Programa Académico _____

DEDICATORIA

Este Documento Es El Producto Final De Mucho Trabajo, Y Esfuerzo Realizado Por Muchas Personas. De Manera Especial Quiero Realizar Una Mención Destacando De Manera Directa En Mi Proceso Formativo Como Ingeniero De Sistemas A Mi Madre. Lucero, A Mi Abuelita Rosalba Y A Todas Las Personas Que De Manera Indirecta Aportaron En Mi Crecimiento Educativo & Como Persona Integra en Valores & Principios.

En Particular Agradezco El Acompañamiento Brindado Por La Universidad Corporación Universitaria Minuto De Dios El Cual Permitió La Consolidación De Competencias Técnicas Necesarias Para El Desarrollo De Esta Investigación, Proyecto De Grado, Aplicación De Producto Final Software Desarrollado, & Tesis Argumentativa

Finalmente, Este Trabajo Fortalece La Práctica Profesional Fundamentada En El Rigor Científico, La Reflexión Crítica, El Compromiso Y La Aplicación De Conocimientos En El Campo De La Ingeniería De Sistemas E Ingeniería De Software. Responsable De Procesos En La Transformación Tecnológica Contemporánea

AGRADECIMIENTOS

Agradezco A Dios; Jesús Rey De Reyes, Señor De Señores. El Haberme Permitido Obtener La Gracia De La Vida, La Inteligencia, Y La Posibilidad De Obtener Estos Conocimientos Respaldados En La Creación De Este Documento Científico Tesis De Grado Y Proyecto De Investigación.

Realmente Destaco La Participación De Mi Hermosa Madre, En Mi Formación Académica Apoyando De Múltiples Maneras Mi Carrera Profesional, Todo Fue Posible Gracias Al Amor Inigualable Que Tiene Una Madre Por Sus Hijos Al Acompañamiento De Mi Familia Y A La Universidad Corporación Universitaria Minuto De Dios En El Cual Curso Mi Pregrado En Ingeniería De Sistemas.

Estableciendo Las Condiciones Evaluativas Para Realizarse Este Proceso Académico Lo Cual Fortalece, La Estructura Argumentativa, Y La Aplicación De Principios Éticos, De Investigación Propios De La Ingeniería De Sistemas, E Ingeniería De Software Avanzado.

Utilizando IA Como Fuente De Optimización
De Procesos En El Desarrollo De Este Documento. Entregando Un Producto Revisado Y Analizado Por Esta Tecnología Clave Y Fundamental En El Proceso Académico Como Ingeniero De Sistemas.

Gracias Dios Por Mi Hermosa Madre Lucero, Mi Abuelita Rosalba, & A La Virgen María Por Aceptar La Voluntad De Dios Y Decir Que Si Al Plan De Salvación, A Ti Dios De La Vida Por El Amor Y La Misericordia Que Tienes Por Las Creaturas Y Toda Mi Familia. Te Amo Dios, Gracias Infinitas**Preliminares**

1. INTRODUCCIÓN

1.1 Contexto del Proyecto

El sector de las dulcerías y pequeños comercios de confitería en Colombia atraviesa un proceso de transformación digital tardío pero inevitable. Con más de 15,000 establecimientos estimados por el DANE (2023) operando bajo procesos manuales o soluciones genéricas inadecuadas, la brecha entre necesidad operativa real y capacidad tecnológica instalada constituye un problema de ingeniería con consecuencias económicas cuantificables: pérdidas por vencimiento de productos, errores de facturación, desabastecimiento no detectado y ausencia total de analítica para la toma de decisiones.

El Sistema de Gestión de Dulcerías (SGDD), operado bajo la marca Dulceia Technologies, nace como respuesta de ingeniería de software a esta realidad sectorial. Su diseño incorpora los principios más rigurosos de la arquitectura de software contemporánea —Clean Architecture, Domain Driven Design, principios SOLID, patrones Repository y DTO— y los eleva con conformidad explícita a estándares internacionales de calidad, seguridad y gestión de requisitos.

1.2 Propósito del Documento

El presente documento cumple la función dual de: (1) Especificación de Requisitos de Software (SRS) conforme a ISO/IEC/IEEE 29148, que establece de forma formal, verificable y trazable todos los requisitos funcionales, no funcionales y de seguridad del sistema; y (2) Informe Técnico de Ingeniería y Auditoría, que documenta el estado de implementación, las decisiones arquitecturales, los resultados de auditoría y las proyecciones del proyecto SGDD v4.0.0.

> Referencia Normativa — ISO/IEC/IEEE 29148:2018
>
> Un SRS conforme a este estándar debe garantizar que todos los requisitos sean: Correctos, No ambiguos,
>
> Completos, Consistentes, Verificables, Trazables y Modificables bajo control formal de cambios.
>
> El presente documento ha sido estructurado para cumplir con todos estos atributos de calidad documental.

1.3 Convenciones del Documento

Los términos 'Deberá' e 'Debe' indican requisito obligatorio contractual. El término 'Podrá' indica capacidad opcional o futura. Cada requisito posee identificador único (RF-XX, RNF-XX, RSEG-XX), criterio de aceptación cuantificable y trazabilidad hacia objetivo de negocio, caso de uso, caso de prueba y estándar normativo correspondiente. Los estados de implementación se codifican como: COMPLETO, EN CURSO, PENDIENTE, BASE.

1.4 Alcance del Sistema Dulceia

Dulceia es una plataforma SaaS multi-tenant orientada a la transformación digital del sector minorista de dulcerías mediante automatización operativa, analítica inteligente y optimización estratégica basada en datos. El sistema opera bajo modelo cloud, permitiendo acceso remoto mediante navegador web sin instalación local en el establecimiento del tenant.

Dulceia NO incluye en su alcance inicial:

- Hardware físico POS (Point of Sale) ni periféricos (impresoras, lectores de código de barras).
- Infraestructura local de red o servidores en sitio en los establecimientos de los tenants.
- Servicios logísticos de distribución externa o transporte.
- Procesamiento bancario directo (se delega a APIs externas certificadas: Stripe, PSE).
- Administración contable formal externa (SIIGO, World Office, etc.).
- Integración obligatoria con sistemas ERP de terceros en fase inicial.
- Desarrollo de aplicación móvil nativa (iOS/Android) en versión 4.0.0.

2. PLANTEAMIENTO DEL PROBLEMA

2.1 Diagnóstico del Sector

El análisis técnico del entorno operativo de las dulcerías colombianas identifica seis categorías de problemáticas que generan pérdidas económicas cuantificables y limitan la competitividad del sector. Este diagnóstico, alineado con el análisis de factores externos requerido por la Cláusula 4.1 de ISO 9001, fundamenta formalmente la necesidad del sistema.

Categoría de Problema	Manifestación Operativa	Impacto Económico Estimado
Gestión de Inventario	Control manual por cuaderno o Excel sin trazabilidad de lotes ni vencimientos.	Pérdida de $300K-$800K COP/mes por caducidad.
Proceso de Ventas	Facturación artesanal con calculadora; sin historial digital ni conciliación automática.	Errores en el 5% de transacciones. Tiempo excesivo en cierre de caja.
Trazabilidad de Datos	Información dispersa en múltiples formatos no integrados entre sí.	Imposibilidad de auditoría interna. Decisiones sin base de datos.
Analítica de Negocio	Ausencia total de reportes automáticos de rentabilidad, tendencias o proyecciones.	Compras reactivas con sobre o sub-stock crónico.
Costos de Adopción	Soluciones ERP existentes (SAP, Odoo) superan la capacidad de inversión del sector.	El 78% de microcomercianates no puede acceder a sistemas ERP estándar.
Gestión de Perecederos	Sin método FIFO automatizado ni alertas preventivas de caducidad por lote.	Rotación de inventario promedio: 3 veces/año vs. 6 posibles con sistema.

2.2 Formulación Formal del Problema

PROBLEMA CENTRAL (conforme a ISO/IEC/IEEE 29148 — Sección de Propósito):

¿Cómo diseñar e implementar un sistema de gestión integral basado en arquitectura SaaS multi-tenant que permita a las dulcerías colombianas automatizar sus procesos administrativos,
controlar inventario con trazabilidad por lote (FIFO), digitalizar ventas, gestionar clientes y proveedores en tiempo real, y acceder a análisis predictivos con IA, en una plataforma técnicamente conforme a estándares ISO, accesible, escalable y segura por diseño?

2.3 Preguntas de Investigación Trazables

1. ¿Qué arquitectura de software garantiza simultáneamente escalabilidad, mantenibilidad y aislamiento de datos para un sistema SaaS multi-tenant conforme a ISO/IEC 15288?
2. ¿Cómo implementar Multi-Tenancy robusto con aislamiento lógico por Tenant_Id sin comprometer el rendimiento definido en RNF-01 (≤ 3 seg, ¿P95)?
3. ¿Qué métricas de calidad del producto (ISO/IEC 25010) son aplicables y medibles en el contexto específico de una plataforma SaaS para dulcerías?
4. ¿Cómo integrar modelos NLP de clasificación de intenciones con precisión mínima del 80% (F1-Score) para automatizar pedidos conversacionales?
5. ¿Qué controles de seguridad técnica (ISO/IEC 27001, Anexo A) son críticos para proteger datos de múltiples tenants en infraestructura compartida?

3. OBJETIVOS

3.1 Objetivo General

Diseñar, implementar, documentar y auditar el Sistema de Gestión de Dulcerías (SGDD / Dulceia)

como plataforma SaaS multi-tenant de nivel empresarial, basada en Clean Architecture y DDD,

que integre módulos completos de gestión operativa, analítica predictiva con IA, autenticación

Zero Trust y control de inventario FIFO por lote, conforme a los estándares internacionales

ISO/IEC/IEEE 29148, ISO/IEC 25010, ISO/IEC 27001 e ISO/IEC 12207.

3.2 Objetivos Específicos con Métricas de Cumplimiento

ID	Objetivo Específico	Métrica de Éxito	Estado
OE-01	Diseñar arquitectura DDD de 4 capas conforme a ISO/IEC 15288 con dependencias hacia el dominio.	59 entidades, 4 capas independientes.	COMPLETO
OE-02	Implementar backend FastAPI con endpoints REST versionados, documentación OpenAPI y manejo de errores HTTP estandarizado.	69 endpoints en /api/v1.	COMPLETO
OE-03	Desarrollar 59 entidades de dominio con invariantes de negocio, valué objects y enums de estado.	59 entidades con reglas formales.	COMPLETO
OE-04	Construir 62 repositorios SQLAlchemy con CRUD completo, filtro obligatorio por tenant_id e índices optimizados.	62 repos, 0 consultas sin tenant_id.	COMPLETO
OE-05	Crear 59 DTOs Pydantic con validaciones formales y 56 servicios de aplicación que orquesten los casos de uso.	115 artefactos en capa aplicación.	COMPLETO
OE-06	Implementar autenticación JWT con modelo Zero Trust, RBAC con 4 roles y validación de tenant en cada petición.	Middleware activo en 100% de rutas.	COMPLETO

OE-07	Desarrollar módulo NLP para bot conversacional con clasificación de intenciones (precisión ≥ 80%).	F1-Score ≥ 0.80 en 8 intenciones.	BASE
OE-08	Crear frontend React 18 con CSS Modules, routing protegido, estado Zustand y conexión a API.	12 páginas, login + dashboard.	60%
OE-09	Integrar PostgreSQL 18.3 con caché Redis 7, rate limiting y backups automáticos.	Contenedores Docker activos 8/8.	COMPLETO
OE-10	Ejecutar y certificar auditoría técnica completa del sistema conforme a criterios ISO/IEC 25010.	8/8 criterios verificados.	COMPLETO

4. JUSTIFICACIÓN

4.1 Justificación Técnica — Decisiones Arquitecturales

La selección de FastAPI como framework backend se sustenta en métricas objetivas de rendimiento: benchmarks independientes (TechEmpower, 2024) posicionan a FastAPI en el percentil 95 de frameworks web para Python, con capacidad de procesar más de 50,000 solicitudes por segundo en configuraciones optimizadas. Su integración nativa con Pydantic garantiza validación de tipos en tiempo de ejecución, eliminando una categoría completa de bugs en producción. La documentación OpenAPI automática (/docs) cumple directamente el atributo de Usabilidad definido en ISO/IEC 25010 Sección 4.1.4.

La adopción de Domain Driven Design responde a la complejidad inherente del dominio de dulcerías: 14 entidades centrales con relaciones complejas, reglas de negocio específicas (FIFO, stock mínimo, control de caducidad, multi-tenancy) y necesidad de un lenguaje ubicuo compartido entre el modelo de software y el dominio del negocio. Con 59 entidades de dominio implementadas, SGDD representa el modelo de dominio más completo documentado para el sector confitero en Colombia.

Referencia Normativa: *ISO/IEC 25010 §4.1.1: Adecuación Funcional — La arquitectura DDD garantiza que el sistema modele correctamente las capacidades y funciones del dominio de negocio objetivo.*

4.2 Justificación Normativa — Cumplimiento de Estándares

La alineación del sistema con once estándares internacionales ISO/IEC no es cosmética: cada norma resuelve un riesgo técnico específico. ISO/IEC 27001 estructura la gestión de seguridad de la información en una arquitectura multi-tenant donde una brecha de aislamiento entre tenants constituiría una falla crítica. ISO/IEC/IEEE 29148 garantiza que los requisitos del sistema sean verificables, eliminando la ambigüedad que tipicamente genera conflictos en proyectos de software. ISO/IEC 25010 convierte atributos subjetivos de calidad (rendimiento, usabilidad) en métricas cuantificables auditables.

4.3 Justificación Económica — Análisis de Viabilidad

Con un costo de desarrollo estimado de $7.5M COP y costos operativos de $440K COP/mes, el modelo de suscripción proyecta ingresos de $6.5M COP/mes al alcanzar 80 clientes (mes 6), generando un ROI acumulado de $28.9M COP en seis meses. Para cada dulcería usuaria, el sistema genera reducción del 75% en tiempo de gestión de inventario, 80% en errores de facturación y eliminación estimada del 60% de pérdidas por caducidad.

4.4 Justificación Social

SGDD democratiza el acceso a tecnología de gestión empresarial para el sector de microcomercianates del confitero colombiano, históricamente excluido de la transformación digital por los costos prohibitivos de los sistemas ERP tradicionales. La plataforma facilita la formalización tributaria, el cumplimiento de la facturación electrónica DIAN y el acceso a analítica predictiva anteriormente exclusiva de grandes retailers.

5. ALCANCES Y LIMITACIONES

5.1 Alcances del Sistema — Conformes a Boundary Definition (ISO/IEC/IEEE 29148 §1.2.4)

5.1.1 Módulos de Backend — Estado: COMPLETO

Módulo	Funcionalidades Incluidas	RF Asociados
Autenticación & RBAC	Login JWT, registro de usuarios, 4 roles (Admin/Vendedor/Operador/Supervisor), renovación de token, bloqueo tras 5 intentos fallidos.	RF-01, RSEG-01
Multi-Tenancy	Aislamiento por tenant_id en 100% de consultas, middleware de validación por petición, límites de recursos por plan.	RMT-01 a RMT-05
Inventario FIFO	CRUD de productos, control por lote, fechas de vencimiento, alertas automáticas de stock mínimo, método FIFO obligatorio.	RF-04, RF-05, RF-06
Ventas & POS	Registro de transacciones, actualización automática de inventario ≤ 2s, múltiples métodos de pago, comprobante digital, rollback en fallo.	RF-09, RF-10
Clientes & Proveedores	CRUD, historial de compras, crédito, segmentación, órdenes de compra, términos de pago.	RF-11, RF-12
Reportes & KPIs	Ventas por período, top productos, rentabilidad, inventario crítico, dashboard de métricas.	RF-13, RF-14
Bot NLP	Pipeline NLP: clasificador de intenciones, extracción de entidades, 8 intenciones, endpoint POST /api/v1/bot/query.	RF-15 (base)
Suscripciones SaaS	3 planes (Básico/Profesional/Empresarial), gestión de límites, estado de suscripción, integración Stripe (diseñada).	RB-05

5.2 Limitaciones — Exclusiones Formales

Limitación	Justificación Técnica	Impacto en Riesgo
Bot NLP sin entrenamiento completo	Requiere dataset mínimo de 500 ejemplos etiquetados por intención con datos reales del sector.	Medio
Facturación DIAN no implementada	Requiere certificación con operador tecnológico DIAN y formato XML UBL 2.1.	Medio
Frontend al 60%	Páginas de negocio (productos, ventas, clientes) pendientes de conexión con API real.	Bajo
Sin pruebas de carga ejecutadas	Pruebas con Locust/JMeter pendientes. RNF-01 (≤3s, 500 usuarios) no validado en condiciones reales.	Alto
SSL/HTTPS no configurado	Requisito crítico para producción. Certificado Let's Encrypt pendiente de configuración en Nginx.	Crítico
Stripe sin integración activa	Arquitectura diseñada y webhook estructurado; integración real pendiente de cuenta de producción.	Bajo

6. MARCO TEÓRICO

6.1 Clean Architecture — Fundamento Estructural

La Arquitectura Limpia, sistematizada por Robert C. Martin (2017), establece que las dependencias de código fuente únicamente pueden apuntar hacia adentro: hacia la capa de mayor abstracción (el Dominio). Esta regla, conocida como la Regla de Dependencia, garantiza que el núcleo del sistema —las reglas de negocio— permanezca completamente independiente de frameworks, bases de datos e interfaces de usuario. El resultado es un sistema donde cambiar PostgreSQL por MongoDB, FastAPI por Django, o React por Vue no requiere modificar ni una línea de lógica de negocio.

En SGDD, las cuatro capas se materializan como: Dominio (entidades Python puras sin dependencias), Aplicación (servicios que orquestan casos de uso), Infraestructura (repositorios SQLAlchemy, clientes Redis, modelos NLP) y Presentación (endpoints FastAPI y componentes React). La violación de esta arquitectura se detecta inmediatamente: si una entidad de dominio importa SQLAlchemy, se ha roto la Regla de Dependencia.

Referencia Normativa: *ISO/IEC 15288:2023 §6.4.2: La arquitectura del sistema debe definir componentes, interfaces, responsabilidades y restricciones. La Clean Architecture cumple todos estos requisitos formalmente.*

6.2 Domain Driven Design (DDD) — Modelo de Negocio

Eric Evans (2003) define DDD como la metodología para abordar complejidad del software conectando la implementación técnica con un modelo evolutivo del dominio de negocio. Sus elementos estructurales en SGDD son: 59 Entidades (objetos con identidad única y ciclo de vida), 3 Value Objects (Dinero, Email, Dirección, inmutables y comparados por atributos), 12 Agregados (clusters tratados como unidad transaccional), 62 Repositorios (abstracciones de colección para recuperar agregados) y 56 Servicios de Dominio (operaciones que no pertenecen naturalmente a una entidad específica).

La invariante de dominio más crítica en Dulceia es el aislamiento de tenant: ninguna entidad puede existir sin un tenant_id válido, y ningún repositorio puede ejecutar una consulta sin filtrar por ese campo. Esta invariante se aplica en la capa de repositorio (infraestructura), no en el dominio, preservando la pureza del modelo de negocio.

6.3 Marco Normativo ISO — Fundamento de Calidad y Seguridad

Estándar	Alcance	Aplicación Específica en SGDD
ISO/IEC/IEEE 29148	Ingeniería de Requisitos	Estructura del SRS, identificadores únicos RF/RNF/RSEG, trazabilidad bidireccional, gestión de cambios.
ISO/IEC 25010	Calidad del Producto Software	8 características: funcionalidad, rendimiento, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad, portabilidad.
ISO/IEC 27001	Seguridad de la Información (SGSI)	RBAC, JWT Zero Trust, cifrado TLS, auditoría de accesos, gestión de incidentes, inventario de activos.
ISO/IEC 27002	Controles de Seguridad	Controles específicos: hash bcrypt, bloqueo por intentos fallidos, segregación de funciones.
ISO/IEC 12207	Ciclo de Vida del Software	Gestión de configuración, control de versiones Git, registro de cambios, validación formal por fase.
ISO/IEC 15288	Ingeniería de Sistemas	Definición de stakeholders, gestión de requisitos del sistema, evaluación de impacto organizacional.
ISO/IEC 29119	Pruebas de Software	Plan maestro de pruebas, casos estructurados, gestión de defectos, validación de aislamiento multi-tenant.
ISO 9001	Gestión de Calidad	Ciclo PDCA, política de calidad, KPIs medibles, procedimiento de no conformidades.
ISO/IEC 27701	Privacidad de Datos	Minimización de datos, consentimiento informado, políticas de retención, derecho de eliminación.
ISO/IEC 38500	Gobierno de TI	Responsabilidad organizacional, gestión estratégica del SaaS, alineación TI-negocio.

6.4 Tecnologías Base — Justificación Técnica

6.4.1 FastAPI (Backend)

Framework asíncrono de alto rendimiento para Python, basado en Starlette y Uvicorn. Seleccionado por: (1) Soporte nativo para async/await, reduciendo la latencia de I/O en operaciones de base de datos. (2) Validación automática con Pydantic, eliminando una clase completa de bugs de tipo en producción. (3) Generación de documentación OpenAPI sin configuración adicional. (4) Benchmarks de rendimiento comparables a Node.js (20,000–50,000 req/s en configuración optimizada).

6.4.2 PostgreSQL 18.3 (Persistencia)

Motor de base de datos relacional seleccionado por: soporte JSONB para datos semiestructurados, funciones de ventana para analítica en SQL, índices GIN para búsquedas de texto completo, ACID estricto garantizando integridad transaccional en operaciones de venta, y conformidad con SQL:2016. El patrón de multi-tenancy con tenant_id indexado en todas las tablas permite consultas en tiempo O (log n) incluso con millones de registros.

6.4.3 Redis 7 (Caché & Rate Limiting)

Almacén de datos en memoria utilizado para: (1) Caché de consultas frecuentes (lista de productos, KPIs del dashboard) con TTL configurable por tipo de dato. (2) Rate limiting distribuido para implementar el control de intentos fallidos de login (RSEG-03). (3) Almacenamiento de sesiones y tokens en lista negra (blacklist JWT). Su integración es crítica para alcanzar el RNF-01 (tiempo de respuesta $\leq$ 3 segundos bajo 500 usuarios concurrentes).

7. ESTADO DEL ARTE

7.1 Análisis Competitivo Técnico

Sistema	Tipo	Arquitectura	Fortalezas Técnicas	Brechas vs SGDD
SAP Business One	ERP Global	Monolítico / SOA	Robusto, completo, integrado con SAP.	Sin vertical dulcerías, sin IA nativa, >$50M COP.
Odoo 17	ERP Open Source	Modular / REST API	Modular, comunidad activa, API bien documentada.	Sin control FIFO perecederos, sin NLP, curva alta.
Shopify	E-commerce SaaS	Microservicios	Escalabilidad probada, pagos integrados, UI excelente.	Sin inventario por lote, sin multi-tenant B2B, sin NLP.
Square POS	POS + Pagos	Cloud / SaaS	Integración hardware, pagos en tiempo real.	No opera en Colombia, sin analítica predictiva.
Siesa Enterprise	ERP Colombiano	Monolítico	Localizado DIAN, cumplimiento fiscal colombiano.	Monolítico, sin SaaS real, costoso para PYMES.
SGDD Dulceia v4.0	SaaS Vertical	DDD / Clean Arch	DDD, IA integrada, FIFO, NLP, ISO alineado, OpenSource.	Frontend 60%, NLP en entrenamiento, sin DIAN aún.

7.2 Comparativa de Características Técnicas Diferenciales

Característica Técnica	SAP	Odoo	Shopify	Siesa	SGDD	Notas
Clean Architecture + DDD	✗	✗	✗	✗	✓	Único con DDD formal
Multi-Tenant SaaS con tenant_id	✗	~	✓	✗	✓	SGDD: modelo propio
Control FIFO por Lote + Caducidad	✓	✓	✗	✓	✓	SGDD: específico dulcerías
Bot NLP Conversacional integrado	✗	✗	✗	✗	~	SGDD: en entrenamiento
Analítica Predictiva Demand.	✓	~	✗	~	~	SGDD: Holt-Winters
Alineación ISO/IEC 25010	~	~	✗	~	✓	SGDD: explícito
Open Source (MIT)	✗	✓	✗	✗	✓	
Costo accesible para PYMES Colombia	✗	~	~	✗	✓	Desde $50K COP/mes

7.3 Tendencias Relevantes (2023-2026)

- Intelligent ERP: convergencia de sistemas de gestión con LLMs (Large Language Models) para automatización de procesos. SGDD anticipa esta tendencia con el bot NLP.
- Vertical SaaS: crecimiento del mercado de software vertical para sectores específicos vs. ERP horizontal genérico. SGDD es pionero en el segmento dulcero colombiano.
- DevSecOps masivo: integración de seguridad como componente transversal en el ciclo de vida. SGDD implementa esta cultura con GitHub Actions y middleware de seguridad en cada petición.
- FastAPI como estándar Python: adopción masiva de FastAPI (18M descargas/mes en 2025) como framework de referencia para APIs de alto rendimiento en Python, validando la decisión arquitectural de SGDD.
-

8. MARCO CONCEPTUAL

8.1 Glosario Técnico Formal (conforme a ISO/IEC/IEEE 29148 §1.3)

De acuerdo con los lineamientos establecidos por ISO/IEC/IEEE 29148, toda SRS debe definir formalmente los términos técnicos empleados para garantizar interpretación uniforme entre stakeholders, reducir riesgo de ambigüedad contractual y facilitar auditorías técnicas.

Término / Acrónimo	Definición Formal en el Contexto de SGDD / Dulceia	Referencia
SaaS	Software as a Service: modelo en que SGDD se aloja en infraestructura cloud administrada y es accesible mediante suscripción sin instalación local en el tenant.	NIST SP 800-145
Multi-Tenant	Arquitectura donde múltiples organizaciones independientes (dulcerías) comparten una instancia de aplicación con aislamiento lógico garantizado por tenant_id.	ISO/IEC 17788
Tenant / Inquilino	Entidad organizacional (dulcería) registrada en el sistema con datos, usuarios,	SRS-DIA-001 §1.3

	configuraciones y operaciones completamente aislados de otros tenants.	
JWT (JSON Web Token)	Token compacto firmado digitalmente (RFC 7519) que contiene identidad del usuario, rol asignado y tenant_id. Válido por 30 minutos con renovación mediante refresh token.	RFC 7519
Zero Trust	Modelo de seguridad 'nunca confiar, siempre verificar': cada petición HTTP debe ser autenticada, autorizada y filtrada por tenant_id independientemente de su origen.	NIST SP 800-207
RBAC	Role-Based Access Control: control de acceso basado en roles. SGDD implementa 4 roles: Administrador, Vendedor, Operador y Supervisor, con permisos específicos por operación.	NIST SP 800-162
FIFO	First In, First Out: método de rotación de inventario donde el primer lote ingresado es el primero en salir. Obligatorio en SGDD para todos los productos con fecha de vencimiento.	RF-05, RF-09
NLP	Natural Language Processing: conjunto de técnicas de IA para interpretar lenguaje humano. En SGDD, se usa para clasificar intenciones del usuario del bot y extraer entidades (producto, cantidad).	RF-15
MAPE	Mean Absolute Percentage Error: métrica de error para modelos de pronóstico. SGDD define MAPE $< 20\%$ como umbral de aceptación para el módulo de predicción de demanda.	Secc. 14
ROP	Reorder Point (Punto de Reorden): nivel de stock en el que debe iniciarse una orden de compra, calculado como $ROP = \bar{d} \cdot L + SS$. Generado automáticamente por el sistema.	Secc. 14.4
DDD	Domain-Driven Design: metodología de diseño centrada en el dominio del negocio.	Evans, 2003

	En SGDD: 59 entidades, 3 value objects, 12 agregados, 62 repositorios.	
DTO	Data Transfer Object: objeto Pydantic que valida y transporta datos entre la API y los servicios de aplicación sin exponer el modelo de dominio. SGDD: 59 DTOs.	Fowler, 2002
SRS	Software Requirements Specification: documento formal que define el comportamiento esperado del sistema conforme a ISO/IEC/IEEE 29148. Este documento es el SRS de SGDD.	ISO 29148
DevSecOps	Práctica que integra seguridad como componente transversal en todo el ciclo de desarrollo, desde el diseño hasta el despliegue. SGDD: GitHub Actions + middleware de seguridad.	ISO/IEC 27001

9. MARCO CONTEXTUAL

9.1 Factores Externos (ISO 9001 Cláusula 4.1)

Conforme a la Cláusula 4.1 de ISO 9001 y la Cláusula 4.1 de ISO/IEC 27001, la organización debe determinar los factores externos que influyen en su propósito estratégico y en su capacidad para lograr los resultados previstos.

9.1.1 Digitalización Acelerada del Comercio Minorista

El sector minorista colombiano atraviesa un proceso de transformación digital impulsado por regulación (facturación electrónica DIAN obligatoria desde 2021), competencia de plataformas digitales y cambios en el comportamiento del consumidor post-pandemia. Sin embargo, la brecha tecnológica entre grandes retailers y microcomercianates persiste: mientras Éxito o Jumbo operan con sistemas ERP integrados con IA, el 78% de dulcerías opera con registro manual o hojas de cálculo. Este entorno exige que Dulceia diseñe una plataforma de adopción mínima, maximizando el valor desde el primer día de uso.

9.1.2 Regulación de Datos Personales en Colombia

La Ley 1581 de 2012 (Habeas Data) y el Decreto 1377 de 2013 establecen obligaciones de confidencialidad, consentimiento informado, minimización de datos y mecanismos de eliminación. En una arquitectura multi-tenant donde los datos de múltiples personas naturales (clientes de las dulcerías) se almacenan en infraestructura compartida, el incumplimiento de estas normas constituye un riesgo legal de alta probabilidad de impacto. El diseño de SGDD implementa estos controles como requisitos de seguridad de primera clase (RSEG-XX).

9.2 Factores Internos (ISO 9001 Cláusula 4.1)

Factor Interno	Descripción Técnica	Impacto en el Sistema
Arquitectura de Microservicios	Cada dominio funcional (auth, inventario, ventas, analítica) es un servicio independiente con API propia.	Escalabilidad selectiva. Despliegue independiente por servicio.

Cultura DevSecOps	Seguridad integrada en todo el ciclo de vida: análisis estático de código, escaneo de dependencias, CI/CD automatizado.	Reducción de vulnerabilidades en producción. Trazabilidad entre código y despliegue.
IA como Diferenciador Central	La analítica predictiva y el bot NLP son componentes estratégicos, no accesorios. Requieren calidad de datos y evaluación continua de modelos.	Valor diferencial demostrable frente a competidores del segmento.
Orientación a Certificación ISO	El proyecto documenta evidencias de cumplimiento para futura certificación ISO 9001 / ISO 27001.	Documentación formal. Auditorías internas. KPIs medibles.
Equipo Unipersonal Especializado	Un desarrollador full stack cubre arquitectura, backend, frontend, DevOps y auditoría.	Alta velocidad de decisión. Riesgo de bus factor = 1. Documentación como mitigación.

9.3 Stakeholders (ISO 9001 Cláusula 4.2)

Conforme a la Cláusula 4.2 de ISO 9001, se identifican las partes interesadas pertinentes, sus necesidades y expectativas que se convierten en requisitos del sistema:

- Dueños de Dulcerías (Tenants): expectativas de reducción de pérdidas por caducidad, reportes estratégicos y alta disponibilidad (≥99.5%). Nivel de influencia: ALTO — son la fuente primaria de ingresos.
- Cajeros y Operadores: expectativas de interfaz intuitiva, velocidad en registro (< 2s) y mínima curva de aprendizaje (≤ 2 horas). Nivel de influencia: ALTO — son los usuarios diarios del sistema.
- Administradores del Sistema: expectativas de RBAC granular, auditoría de operaciones y backups confiables. Nivel de influencia: MEDIO.
- Evaluadores Académicos: expectativas de rigor técnico, alineación normativa, documentación SRS y trazabilidad. Nivel de influencia: ALTO en el contexto académico.
- Equipo de Pruebas / Auditores: expectativas de casos de prueba estructurados, evidencia de validación y gestión de defectos. Nivel de influencia: MEDIO.

10. MARCO LEGAL Y NORMATIVO

10.1 Normatividad Colombiana Aplicable

Norma	Descripción	Aplicación Específica en SGDD	Riesgo por Incumplimiento
Ley 1581/2012	Protección de Datos Personales (Habeas Data). Obligaciones de confidencialidad, consentimiento y minimización.	Hash de contraseñas, cifrado TLS, política de retención, consentimiento en registro.	Crítico
Decreto 1377/2013	Reglamentación del tratamiento de datos personales y habeas data.	Política de privacidad, autorización explícita para uso de datos de clientes.	Alto
Res. DIAN 000042	Facturación electrónica obligatoria con formato XML UBL 2.1 y operador tecnológico.	Módulo diseñado arquitecturalmente. Integración XML pendiente. No en alcance v4.0.	Medio
Ley 527/1999	Comercio electrónico: validez legal de mensajes de datos y firmas digitales.	Validez jurídica de transacciones digitales procesadas por el sistema.	Bajo
Decreto 1074/2015	Regulación general del sector comercio. Normas para establecimientos comerciales.	Marco normativo general para las dulcerías como establecimientos de comercio.	Bajo

10.2 Marco Normativo Internacional ISO — Aplicación en SGDD

El Sistema de Gestión de Dulcerías se desarrolla bajo alineación integral con once estándares internacionales ISO/IEC. Esta alineación no constituye certificación formal, sino conformidad normativa conceptual y estructural que posiciona el proyecto como arquitectura de nivel corporativo. El detalle de aplicación de cada estándar se referencia en las secciones correspondientes del presente documento.

CONCLUSIÓN NORMATIVA DE AUDITORÍA:

SGDD demuestra conformidad estructural con: ISO/IEC/IEEE 29148 (Requisitos) · ISO/IEC 25010 (Calidad)

ISO/IEC 27001 (Seguridad) · ISO/IEC 27002 (Controles) · ISO/IEC 12207 (Ciclo de Vida)

ISO/IEC 15288 (Sistemas) · ISO/IEC 29119 (Pruebas) · ISO 9001 (Calidad Org.) · ISO/IEC 38500 (Gobierno TI)

Esta alineación eleva el proyecto de una tesis académica a una arquitectura alineada a estándares corporativos.

10.3 Licencia y Propiedad Intelectual

El código fuente de SGDD está protegido bajo licencia MIT (Massachusetts Institute of Technology), que permite uso comercial libre, modificación, distribución e inclusión en proyectos propietarios, con el único requisito de conservar el aviso de copyright. Todas las dependencias del stack tecnológico (FastAPI/MIT, React/MIT, SQLAlchemy/MIT, PostgreSQL/PostgreSQL License, Redis/BSD-3) son compatibles con esta licencia para uso comercial.

11. DISEÑO DEL SISTEMA

Referencia Normativa: *ISO/IEC/IEEE 42010:2022 — Descripción Arquitectónica: define componentes, relaciones, responsabilidades, restricciones y entorno operativo.*

11.1 Modelo Arquitectónico — Clean Architecture + DDD

La arquitectura de SGDD implementa el modelo hexagonal (también denominado puertos y adaptadores) con cuatro capas concéntricas, donde la Regla de Dependencia garantiza que las capas externas dependen de las internas, pero nunca al inverso. El dominio en el núcleo no tiene ninguna dependencia de frameworks, bases de datos ni protocolos de red.

REGLA DE DEPENDENCIA (Clean Architecture, Martin 2017):

CAPA 4 — Presentación (React SPA + FastAPI Endpoints) ——► depende de ▼

CAPA 3 — Infraestructura (SQLAlchemy, Redis, NLP, Stripe) ——► depende de ▼

CAPA 2 — Aplicación (Servicios de Aplicación + DTOs) ——► depende de ▼

CAPA 1 — Dominio (Entidades + Value Objects + Reglas) ——► NÚCLEO (sin dependencias)

VERIFICACIÓN DE AUDITORÍA: Si una entidad de dominio importa SQLAlchemy → VIOLACIÓN CRÍTICA.

Estado en SGDD: todas las entidades son Python puro sin dependencias externas. ✓ CONFORME.

11.2 Inventario Completo de Componentes Auditados

Capa	Tecnología	Componente	Cantidad	Estado	ISO 25010
Presentación	React 18 + CSS Modules	Páginas (Login, Dashboard, +10 pendientes)	12	60%	Usabilidad
Presentación	FastAPI + Pydantic	Endpoints REST /api/v1 (19 módulos)	69	COMPLETO	Func. & Rendim.
Aplicación	Python puro	Servicios de aplicación con casos de uso	56	COMPLETO	Mantenib.
Aplicación	Pydantic v2	DTOs (Create, Update, Response por entidad)	59	COMPLETO	Seguridad
Dominio	Python puro (sin deps)	Entidades de dominio con invariantes	59	COMPLETO	Func. Funcional
Dominio	Python puro	Value Objects (Dinero, Email, Dirección)	3	COMPLETO	Fiabilidad
Infraestructura	SQLAlchemy 2.0	Repositorios con CRUD + filtro tenant_id	62	COMPLETO	Portabilidad
Infraestructura	scikit-learn / TF	Modelos NLP, pronóstico, recomendaciones	3	BASE	Func. Func.
Infraestructura	Redis 7	Caché, Rate Limiting, Blacklist JWT	1	COMPLETO	Rendimiento
Seguridad	bcrypt + JWT + Middleware	Auth, RBAC, tenant middleware, rate limiter	4	COMPLETO	Seguridad
Operación	PowerShell 7	Menú MENU_PRINCIPAL.ps1 (41 opciones)	41	COMPLETO	Mantenib.

11.3 Requisitos Funcionales Auditables (RF)

Referencia Normativa: *ISO/IEC/IEEE 29148 — Cada RF incluye: identificador único, descripción formal, entradas, proceso, salidas, criterios de aceptación verificables, prioridad y trazabilidad.*

ID	Nombre del Requisito	Descripción Formal	Criterio de Aceptación Verificable	Prio.	Estado
RF-01	Registro de Usuario	El sistema deberá crear cuentas de usuario con hash bcrypt, RBAC y auditoría en log.	Hash verificable en BD. Evento en log. Correo único por tenant.	Alta	✓
RF-02	Autenticación JWT	El sistema deberá emitir JWT firmado con HS256, incluir tenant_id, expirar en 30 min y bloquear tras 5 intentos fallidos.	Token emitido en < 500ms. Bloqueo activado tras intento 5.	Alta	✓
RF-03	Gestión RBAC	El sistema deberá controlar acceso por 4 roles (Admin/Vendedor/Operador/Supervisor) validando permisos antes de cada operación sensible.	Operación rechazada con HTTP 403 si rol insuficiente.	Alta	✓
RF-04	Gestión de Inventario	El sistema deberá permitir CRUD de productos, con control por lote, fecha de vencimiento, proveedor y alertas de stock mínimo automáticas.	Alerta generada cuando stock < stock_mínimo. Verificable en API.	Alta	✓
RF-05	Rotación FIFO	El sistema deberá aplicar método FIFO obligatorio en todos los productos con fecha de vencimiento al procesar salidas de inventario.	Lote más antiguo siempre se consume primero. Error < 0.1%.	Alta	✓

RF-06	Alertas de Caducidad	El sistema deberá generar alertas automáticas cuando un lote tenga fecha de vencimiento en ≤ 7 días, calculando pérdida estimada.	Alerta generada en ventana de 7 días. Monto de pérdida calculado.	Alta	✓
RF-09	Registro de Venta	El sistema deberá registrar ventas, actualizar inventario automáticamente aplicando FIFO en ≤ 2s y hacer rollback en caso de fallo.	Inventario actualizado ≤ 2s (P95). Rollback verificable.	Alta	✓
RF-13	Reportes Estratégicos	El sistema deberá generar reportes de ventas por período, top productos por volumen, rentabilidad y estado crítico de inventario.	Reporte generado en < 3s. Datos coherentes con BD.	Media	✓
RF-15	Bot Conversacional NLP	El sistema deberá procesar consultas en lenguaje natural, clasificar intenciones con F1-Score ≥ 0.80 y responder con datos reales del tenant.	F1-Score ≥ 0.80 en 8 intenciones. Respuesta < 1.5s.	Media	BASE

11.4 Requisitos No Funcionales (RNF) — Mapeados a ISO/IEC 25010

ID	Categoría ISO 25010	Descripción Formal	Métricas Verificables	Estado	Validación
RNF-01	Eficiencia de Desempeño	El sistema deberá responder a solicitudes en ≤ 3 seg bajo carga nominal.	P95 ≤ 3s. P99 ≤ 5s. CPU ≤ 75% promedio.	PENDIENTE	Load Testing Locust
RNF-02	Fiabilidad / Disponibilidad	El sistema deberá garantizar	Downtime máximo: 43.8	PENDIENTE	Prometheus + SLA

		disponibilidad ≥ 99.5% anual excluyendo mantenimientos programados.	hs/año. Monitoreo 24/7.		
RNF-03	Escalabilidad	El sistema deberá soportar escalado horizontal automático ante incremento ≥ 70% de CPU.	Auto-scaling activo. ≥ 100 tenants. Duplicar instancias en < 5 min.	PARCIAL	Kubernetes / Docker
RNF-04	Usabilidad	El sistema deberá ser aprendible en ≤ 2 horas de capacitación para operadores.	Tasa error < 5% en tareas comunes. Satisfacción ≥ 80% (UAT).	PENDIENTE	Pruebas UAT
RNF-05	Mantenibilidad	El sistema deberá implementarse con cobertura de pruebas unitarias ≥ 70% y documentación actualizada en cada release.	Cobertura ≥ 70%. MTTR críticos ≤ 48 hs.	PENDIENTE	pytest coverage
RNF-06	Seguridad	El sistema deberá cifrar tráfico con TLS 1.3, almacenar contraseñas con bcrypt (cost=12) y registrar eventos de auditoría.	0 contraseñas en texto plano. TLS activo. Logs completos.	PARCIAL	Auditoría técnica
RNF-07	Multi-Tenancy	El sistema deberá garantizar aislamiento total de datos entre tenants: ningún tenant podrá acceder a datos de otro.	0 violaciones de tenant en 1000 pruebas. Middleware activo en 100% de rutas.	COMPLETO	Pruebas integración

11.5 Requisitos de Seguridad (RSEG) — ISO/IEC 27001

ID	Control de Seguridad	Implementación Técnica	Control ISO 27001 / Anexo A
RSEG-01	Autenticación fuerte	JWT HS256, expiración 30 min, refresh token rotativo, validación en cada petición.	A.9.4 Control de acceso a sistemas
RSEG-02	Almacenamiento seguro de credenciales	bcrypt con cost=12 para hash de contraseñas. Prohibido texto plano o MD5.	A.9.4.3 Información de autenticación secreta
RSEG-03	Protección contra fuerza bruta	Rate limiting Redis: bloqueo IP tras 5 intentos fallidos en 10 minutos. Alerta en Grafana.	A.9.4.5 Control de acceso al código fuente
RSEG-04	Cifrado en tránsito	TLS 1.3 obligatorio en producción. HTTP rechazado. Configuración Nginx con cabeceras HSTS.	A.10.1.1 Política de uso de controles criptográficos
RSEG-05	Auditoría de operaciones	Log de todas las operaciones CRUD: quién, qué, cuándo, desde qué IP, en qué tenant.	A.12.4.1 Registro de eventos
RSEG-06	Aislamiento multi-tenant	Middleware valida tenant_id en cada petición. Repositorios filtran obligatoriamente por tenant. Sin bypass posible.	A.13.1.3 Segregación en redes
RSEG-07	Backups y continuidad	Script respaldar-bd.sh ejecutado diariamente. Restore probado mensualmente (opción 14 del menú).	A.12.3.1 Copias de seguridad de información

12. MODELO DE DATOS

12.1 Modelo Entidad-Relación — Tablas Principales

Tabla	Atributos Clave (PK, FK, índices)	Reglas de Negocio y Restricciones	Relaciones
inquilinos	id PK, nombre, subdominio UNIQUE, plan CHECK('basico','profesional','empresarial'), estado, max_usuarios, max_productos	Plan determina límites de recursos. Estado 'suspendido' bloquea todas las operaciones.	1:N usuarios, productos, ventas
usuarios	id PK, nombre, email UNIQUE(tenant), password_hash, rol CHECK('admin','vendedor','operador','supervisor'), activo, tenant_id FK+INDEX	Email único por tenant (no global). Password nunca en texto plano. Rol validado por RBAC.	N:1 inquilinos
productos	id PK, nombre, descripcion, precio DECIMAL(10,2) CHECK(>0), stock INT CHECK(≥0), stock_minimo, categoria_id FK, proveedor_id FK, tenant_id FK+INDEX, activo	Stock no puede ser negativo. Precio mayor que cero. Eliminación lógica (activo=false).	N:1 categorias, proveedores; 1:N lotes, detalles_venta
lotes_inventario	id PK, producto_id FK, cantidad, fecha_ingreso, fecha_vencimiento, proveedor_id FK, tenant_id FK+INDEX	FIFO: lote con fecha_ingreso más antiguo se consume primero. Alertas si vencimiento ≤ 7 días.	N:1 productos

ventas	id PK, cliente_id FK NULL, total DECIMAL(10,2) CHECK(≥0), estado CHECK('pendiente','completada','cancelada'), tenant_id FK+INDEX+fecha	Rollback automático si actualización de stock falla. Estado 'cancelada' no modifica inventario.	1:N detalles_venta, pagos; 1:1 facturas
detalles_venta	id PK, venta_id FK, producto_id FK, lote_id FK, cantidad INT CHECK(>0), precio_unitario, subtotal, tenant_id FK	Subtotal = cantidad × precio_unitario. Lote_id referencia el lote consumido (trazabilidad FIFO).	N:1 ventas, productos, lotes
clientes	id PK, nombre, email, telefono, tipo CHECK('regular','vip','credito'), credito_disponible DECIMAL, tenant_id FK+INDEX, activo	Crédito no puede ser negativo. Eliminación lógica.	1:N ventas
pagos	id PK, venta_id FK, monto DECIMAL(10,2), metodo CHECK('efectivo','tarjeta','transferencia','credito'), estado, fecha_pago	Suma de pagos debe igualar total de venta. Método validado contra catálogo.	N:1 ventas
suscripciones	id PK, inquilino_id FK UNIQUE, plan, fecha_inicio, fecha_vencimiento, estado, stripe_subscription_id	Un inquilino solo puede tener una suscripción activa. Vencida bloquea acceso.	1:1 inquilinos

12.2 Definiciones SQL — Tablas Críticas con Índices

```
-- TABLA CENTRAL MULTI-TENANCY
CREATE TABLE inquilinos (
```

```
    id          SERIAL PRIMARY KEY,
    nombre      VARCHAR (100) NOT NULL,
    subdominio VARCHAR (50) UNIQUE NOT NULL,
    plan        VARCHAR (20) CHECK (plan IN ('basico','profesional','empresarial')) NOT
NULL,
    estado      VARCHAR (20) DEFAULT 'activo',
    max_usuarios INTEGER DEFAULT 5,
    max_productos INTEGER DEFAULT 1000,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- VENTAS CON ÍNDICES OPTIMIZADOS PARA MULTI-TENANT
CREATE TABLE ventas (
    id          SERIAL PRIMARY KEY,
    cliente_id INTEGER REFERENCES clientes(id),
    total       DECIMAL (10,2) NOT NULL CHECK (total >= 0),
    estado      VARCHAR (20) DEFAULT 'pendiente',
    tenant_id   INTEGER NOT NULL REFERENCES inquilinos(id),
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_ventas_tenant      ON ventas(tenant_id);
CREATE INDEX idx_ventas_tenant_fecha ON ventas (tenant_id, fecha_creacion DESC);

-- LOTES CON FIFO Y ALERTAS DE CADUCIDAD
CREATE TABLE lotes_inventario (
    id                  SERIAL PRIMARY KEY,
    producto_id         INTEGER NOT NULL REFERENCES productos(id),
    cantidad            INTEGER NOT NULL CHECK (cantidad >= 0),
    fecha_ingreso       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_vencimiento DATE NOT NULL,
    tenant_id           INTEGER NOT NULL REFERENCES inquilinos(id)
);
CREATE INDEX idx_lotes_fifo ON lotes_inventario (producto_id, fecha_ingreso ASC);
CREATE INDEX idx_lotes_vencimiento ON lotes_inventario (tenant_id,
fecha_vencimiento ASC);
```

12.3 Estadísticas del Modelo de Datos

Concepto	Cantidad	Observación Técnica
Tablas principales PostgreSQL	10	+ tablas auxiliares de auditoría y configuración.
Entidades de dominio DDD (Python puro)	59	Con invariantes de negocio y validaciones de estado.
Repositorios SQLAlchemy	62	100% con filtro obligatorio por tenant_id.
Índices optimizados definidos	15+	Incluyendo índices compuestos (tenant_id, fecha).
Value Objects inmutables	3	Dinero (con moneda), Email (validado), Dirección.
Enums de estado por entidad	10+	Estado de venta, pago, suscripción, usuario, producto.
Relaciones FK entre tablas	15+	Con restricciones de integridad referencial activas.

13. DISEÑO DEL BOT CONVERSACIONAL

13.1 Arquitectura del Pipeline NLP

El módulo de bot conversacional implementa un pipeline de procesamiento de lenguaje natural de cinco etapas secuenciales, desde la entrada del usuario hasta la respuesta enriquecida con datos reales del tenant. La arquitectura sigue el patrón Clasificador de Intenciones + Extractor de Entidades + Orquestador de Dominio, desacoplando el procesamiento lingüístico de la lógica de negocio.

PIPELINE NLP — 5 ETAPAS:

1. PREPROCESAMIENTO: tokenización, normalización (lowercase, acentos), eliminación de stopwords español.
2. VECTORIZACIÓN: TF-IDF con n-gramas (1,2) sobre vocabulario del dominio de dulcerías.
3. CLASIFICACIÓN: modelo entrenado (LinearSVC o LogisticRegression) → intención + confidence score.
4. EXTRACCIÓN DE ENTIDADES: regex + NER para productos, cantidades, fechas y acciones.
5. ORQUESTACIÓN: servicio de dominio ejecuta operación real y genera respuesta contextualizada.

13.2 Matriz de Intenciones y Métricas

Intención	Ejemplos de Entrenamiento	Acción de Sistema	F1-Score Meta	Estado
saludo	'Hola', 'Buenos días', 'Buenas tardes'	Saludo personalizado con nombre del usuario.	≥ 0.90	BASE
consultar_stock	'¿Cuántos chocolates hay?', '¿Tengo masmelos?'	Consulta repositorio inventario del tenant.	≥ 0.80	BASE

consultar_precio	'¿Cuánto cuesta el bom bom bum?', 'Precio de...'	Búsqueda en catálogo de productos del tenant.	≥ 0.80	BASE
ventas_dia	'Ventas de hoy', '¿Cuánto vendí?', 'Reporte del día'	Reporte agregado de ventas del día actual.	≥ 0.85	BASE
productos_populares	'¿Qué se vende más?', 'Top productos'	Top 5 productos por volumen (últimos 30 días).	≥ 0.75	BASE
stock_bajo	'¿Qué necesito reponer?', 'Alertas de inventario'	Productos bajo stock mínimo del tenant.	≥ 0.80	BASE
pronostico_ventas	'¿Cuánto voy a vender?', 'Proyección semana'	Modelo Holt-Winters con datos históricos del tenant.	≥ 0.70	PENDIENTE
ayuda	'¿Qué puedes hacer?', 'Ayuda', 'Comandos'	Lista de capacidades del bot con ejemplos.	≥ 0.95	BASE

13.3 Componentes Implementados y Estado

Componente	Archivo	Función Técnica	Estado
Modelo de intenciones	modelo_intenciones.py	Pipeline TF-IDF + LinearSVC. Serializado en .pkl.	LISTO
Clasificador	clasificador.py	Predicción de intención + confidence score con umbral configurable.	LISTO
Entrenador	entrenador.py	Entrenamiento con dataset etiquetado. Evaluación precision/recall/F1.	LISTO
Endpoint API	api/v1/bot.py	POST /api/v1/bot/query — recibe consulta, retorna	LISTO

		respuesta JSON estructurada.	
Servicio del Bot	bot_servicio.py	Orquesta clasificador + repositorios de datos + generación de respuesta contextual.	LISTO
Dataset de entrenamiento	–	Mínimo 500 ejemplos etiquetados por las 8 intenciones del dominio dulcero.	PENDIENTE
Componente Chat (Frontend)	ChatBot.jsx	Interfaz de chat flotante React con historial de conversación y scroll automático.	PENDIENTE

14. MODELO MATEMÁTICO

14.1 Pronóstico de Ventas — Suavizamiento Exponencial Triple (Holt-Winters)

El módulo de pronóstico implementa el método de Holt-Winters multiplicativo, elegido por su capacidad para capturar simultáneamente tendencia y estacionalidad multiplicativa — ambas presentes en los patrones de venta de dulcerías (mayor demanda en fines de semana, quincenas, días festivos y fechas especiales).

$$F(t+h) = [\, L(t) + h \cdot T(t)] \cdot S\,(t + h - p)$$

Los componentes del modelo se actualizan recursivamente en cada período observado:

Componente	Ecuación de Actualización	Descripción
Nivel L(t)	$L(t) = \alpha \cdot [Y(t)/S(t-p)] + (1-\alpha) \cdot [L(t-1)+T(t-1)]$	Estimación del valor base sin estacionalidad en el período t.
Tendencia T(t)	$T(t) = \beta \cdot [L(t)-L(t-1)] + (1-\beta) \cdot T(t-1)$	Tasa de cambio del nivel entre períodos consecutivos.
Estacionalidad S(t)	$S(t) = \gamma \cdot [Y(t)/L(t)] + (1-\gamma) \cdot S(t-p)$	Factor multiplicativo de estacionalidad del período t del ciclo.
Parámetros α, β, γ	$\alpha, \beta, \gamma \in (0,1)$	Optimizados minimizando MSE sobre datos históricos por tenant. p=7 (ciclo semanal).

La métrica de evaluación principal es el MAPE (Mean Absolute Percentage Error): $MAPE = (100/n) \cdot \Sigma\, |Y(t) - F(t)| \,/\, Y(t)$. El umbral de aceptación del modelo es $MAPE < 20\%$, equivalente a un error promedio inferior al 20% sobre los valores reales de ventas.

14.2 Predicción de Demanda — Regresión Lineal Múltiple

El modelo de predicción de demanda por producto utiliza regresión lineal múltiple como aproximación base, escalable a Gradient Boosting (XGBoost/LightGBM) cuando el volumen de datos históricos por tenant sea suficiente (> 90 días de registro).

$$D = \beta_0 + \beta_1 \cdot P + \beta_2 \cdot E + \beta_3 \cdot T + \beta_4 \cdot C + \epsilon$$

Variable	Descripción	Rango/Unidad	Fuente de Datos
D	Demanda proyectada del producto (variable dependiente).	Unidades / día	Calculada sobre histórico de ventas.
P	Precio del producto en el período de predicción.	COP / unidad	Tabla productos del tenant.
E	Factor de estacionalidad (mes, día de semana, festivo binario).	Índice 0–2	Calendario + histórico de ventas.
T	Tendencia histórica: regresión sobre últimos 90 días de ventas.	Δ unidades/día	Series de tiempo de ventas.
C	Proxy de competencia en la categoría (número de productos sustitutos activos).	Conteo	Tabla categorías del tenant.
$\beta_0 \ldots \beta_4$	Coeficientes estimados por Mínimos Cuadrados Ordinarios (OLS).	–	scikit-learn LinearRegression.
ϵ	Término de error aleatorio con distribución normal $N(0, \sigma^2)$.	–	Asumido; validado con Q-Q plot.

14.3 Motor de Recomendaciones — Filtrado Colaborativo

El motor de recomendaciones implementa similitud coseno entre vectores de comportamiento de usuario (historial de compras) para identificar productos con alta probabilidad de compra conjunta o cruzada, aplicable tanto a clientes recurrentes como a campañas de marketing dirigidas.

$$\text{sim}(u, v) = \Sigma_i (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v) \,/\, [\sqrt{\Sigma_i (r_{ui} - \bar{r}_u)^2} \cdot \sqrt{\Sigma_i (r_{vi} - \bar{r}_v)^2}]$$

La predicción de rating estimado para el producto i y el usuario u se calcula como: $\hat{r}_{ui} = \bar{r}_u + [\Sigma_{v \in N(u)} \text{sim}(u, v) \cdot (r_{vi} - \bar{r}_v)] \,/\, \Sigma_{v \in N(u)} |\text{sim}(u, v)|$, donde N(u) es el conjunto de k usuarios más similares a u (k-NN, k=5 por defecto). El sistema retorna el top-5 de productos con mayor $\hat{r}_{ui}$ como recomendaciones personalizadas.

14.4 Algoritmo de Punto de Reorden (ROP) — Gestión de Stock

El cálculo automático del punto de reorden implementa el modelo de inventario con demanda variable (distribución normal) y tiempo de entrega fijo, siguiendo la teoría clásica de gestión de inventarios:

$$ROP = \bar{d} \cdot L + SS \quad \text{donde} \quad SS = z \cdot \sigma_d \cdot \sqrt{L}$$

Parámetro	Definición	Valor SGDD	Fuente
$\bar{d}$	Demanda diaria promedio del producto.	Calculado: Σ ventas(30 días) / 30	Histórico de detalles_venta.
L	Tiempo de entrega del proveedor en días.	Configurable por proveedor en el tenant.	Tabla proveedores.
SS	Stock de seguridad para absorber variabilidad.	$SS = 1.65 \cdot \sigma_d \cdot \sqrt{L}$	Calculado automáticamente.
z	Factor Z de nivel de servicio deseado.	z = 1.65 (nivel servicio 95%)	Configurable por tenant [1.28, 1.96, 2.05].
σ_d	Desviación estándar de la demanda diaria.	Calculado sobre últimos 30 días.	Histórico de ventas por producto.

Cuando el stock actual de un producto desciende al nivel ROP, el sistema genera automáticamente: (1) alerta de reposición en el dashboard, (2) notificación al rol Supervisor, y (3) propuesta de orden de compra al proveedor habitual con la cantidad óptima EOQ.

15. PLAN EXPERIMENTAL

Referencia Normativa: *ISO/IEC 29119:2013 — Define estructura de Plan Maestro de Pruebas, Diseño de Pruebas, Casos de Prueba y Gestión de Defectos.*

15.1 Estrategia de Pruebas

La estrategia de pruebas de SGDD se estructura en cuatro niveles conforme a ISO/IEC 29119: pruebas unitarias (componentes aislados), pruebas de integración (interacción entre capas), pruebas de sistema (comportamiento end-to-end) y pruebas de aceptación (validación con usuarios reales). Se utilizará la pirámide de pruebas como modelo de distribución: 70% unitarias, 20% integración, 10% sistema.

15.2 Casos de Prueba Estructurados

ID Prueba	Componente	Tipo	Descripción del Caso	Criterio de Éxito	Estado
PT-01	Autenticación	Integración	POST /auth/login con credenciales válidas → JWT emitido con tenant_id correcto.	HTTP 200, JWT válido, tenant_id en payload.	PROB.
PT-02	Autenticación	Seguridad	5 intentos fallidos en 10 min → cuenta bloqueada automáticamente.	HTTP 429 en intento 6. Redis registra el bloqueo.	PEND.
PT-03	Multi-Tenancy	Aislamiento	Usuario de Tenant A intenta acceder a datos del Tenant B usando JWT válido de A.	HTTP 403 en 100% de intentos. 0 datos de B expuestos.	PROB.
PT-04	Inventario FIFO	Funcional	Producto con 3 lotes de fechas distintas → venta consume siempre el lote más antiguo.	Lote FIFO consumido en 100% de pruebas. Error stock < 0.1%.	PEND.

PT-05	Ventas	Rendimiento	Registro de venta con actualización de inventario bajo carga de 50 usuarios concurrentes.	Actualización completada ≤ 2s en P95 de casos.	PEND.
PT-06	Caducidad	Funcional	Lote con fecha_vencimiento = HOY + 5 días → alerta automática generada en dashboard.	Alerta visible en API /alertas con datos correctos del lote.	PEND.
PT-07	Bot NLP	IA	Corpus de 100 consultas etiquetadas → clasificador evalúa F1-Score por intención.	F1-Score ≥ 0.80 en todas las intenciones definidas.	PEND.
PT-08	Dashboard	Sistema	Login completo → carga dashboard con datos reales del tenant → visualización correcta.	Dashboard renderiza en < 3s con datos actualizados.	PROB.
PT-09	Backup / Restore	Continuidad	Ejecución de script respaldar-bd.sh → archivo .dump generado → restauración correcta.	BD restaurado sin pérdida de datos. Hash verificado.	PROB.
PT-10	Rendimiento Global	Carga	500 usuarios concurrentes simulados con Locust durante 10 minutos → métricas de rendimiento.	P95 ≤ 3s. CPU ≤ 75%. 0 errores HTTP 5xx.	PEND.

15.3 Plan de Pruebas por Fase

Fase de Prueba	Herramientas	Cobertura Meta	Criterio de Aprobación

Pruebas Unitarias (pytest)	pytest, pytest-asyncio, pytest-mock	≥ 70% líneas de código	0 tests rotos. Cobertura ≥ 70% verificada con coverage.py.
Pruebas de Integración	httpx TestClient, PostgreSQL test DB	100% endpoints críticos	Todos los endpoints RF retornan HTTP 200/201/204 esperados.
Pruebas de Seguridad	OWASP ZAP, Bandit (análisis estático)	OWASP Top 10	0 vulnerabilidades críticas o altas en reporte ZAP.
Pruebas de Carga	Locust, Apache Bench (ab)	RNF-01: 500 usuarios concurrentes	P95 ≤ 3s. P99 ≤ 5s. Tasa error < 0.1%.
Pruebas de Aceptación (UAT)	Usuarios reales, formulario SUS	5 dulcerías piloto, 3 semanas	SUS Score ≥ 70. Tasa adopción ≥ 80%.
Validación del Bot NLP	sklearn metrics, corpus etiquetado	F1-Score en 8 intenciones	F1-Score global ≥ 0.80. MAPE pronóstico < 20%.

16. METODOLOGÍA DE DESARROLLO

Referencia Normativa: *ISO/IEC 12207:2017 — Procesos del ciclo de vida del software: desarrollo, operación, mantenimiento, gestión de configuración, validación y verificación.*

16.1 Ciclo de Vida — Modelo Ágil Iterativo con Control Documental

SGDD adopta un modelo de desarrollo ágil iterativo con 8 fases secuenciales, donde cada fase produce artefactos auditables y es validada antes de iniciar la siguiente. Este enfoque integra la agilidad de Scrum con el rigor documental requerido por ISO/IEC 12207 y la trazabilidad exigida por ISO/IEC/IEEE 29148.

```
ESTADO DE FASES — SISTEMA SGDD v4.0.0 (Auditado el 08/03/2026)

  [✓] FASE 0: Preparación del entorno, venv, dependencias base, Git
COMPLETADA
  [✓] FASE 1: Docker Compose, PostgreSQL 18.3, Redis 7, env, configuración central
COMPLETADA
  [✓] FASE 2: 59 entidades de dominio DDD, value objects, enums de estado
COMPLETADA
  [✓] FASE 3: 62 repositorios SQLAlchemy, modelos ORM, CRUD + filtros tenant
COMPLETADA
  [✓] FASE 4: 59 DTOs Pydantic v2, 56 servicios de aplicación
COMPLETADA
  [✓] FASE 5: 68 endpoints REST base versionados /api/v1
COMPLETADA
  [✓] FASE 6: 69 endpoints + middlewares (tenant, auth, rate limiting, logging)
COMPLETADA
  [~] FASE 7: Frontend React 18, CSS Modules, conexión API real                60% EN
CURSO
  [] FASE 8: POS, Reportes, DIAN, Bot completo, Stripe, pruebas exhaustivas
PENDIENTE
  [] FASE 9: CI/CD completo, SSL producción, despliegue en cloud
PENDIENTE

  TOTAL AUDITADO: 304 archivos | 1,600+ métodos | 136+ rutas | 8/8 requisitos OK
```

16.2 Herramientas del Ecosistema Técnico

Categoría	Herramientas	Versión	Propósito en SGDD
IDE	Visual Studio Code + extensiones Python, ESLint, Prettier, GitLens	Latest	Desarrollo, depuración y control de calidad de código.
Gestión Versiones	Git 2.53.0, GitHub, GitHub Actions (CI/CD)	2.53.0	Control de versiones, colaboración, pipeline de integración continua.
Backend	Python 3.14.3, FastAPI, Uvicorn, Pydantic v2, SQLAlchemy 2.0, Alembic, bcrypt	Auditada	Core del sistema: API, ORM, validación, seguridad, migraciones.
Frontend	Node.js v24.14.0, React 18, Vite 5, React Router v6, Axios, Zustand, CSS Modules	Auditada	SPA con estado global, routing protegido y caché de peticiones.
Bases de Datos	PostgreSQL 18.3, Redis 7, Alembic (migraciones), DBeaver (exploración)	Auditada	Persistencia relacional, caché en memoria, control de migraciones.
IA / ML	scikit-learn (TF-IDF, LinearSVC, LinearRegression), TensorFlow (LSTM futuro), nltk	Latest	NLP, pronóstico de ventas, predicción de demanda, recomendaciones.
Infraestructura	Docker Desktop, Docker Compose, Nginx, Prometheus, Grafana, GitHub Actions	Latest	Contenerización, proxy reverso, monitoreo de métricas y alertas.
Operación Windows	PowerShell 7, MENU_PRINCIPAL.ps1 (41 opciones)	PS7	Control operacional completo del sistema en entorno de desarrollo.

17. GESTIÓN DE RIESGOS

Referencia Normativa: *ISO/IEC 27005:2022 — Gestión formal de riesgos de seguridad de la información. ISO 9001 Cláusula 6.1 — Acciones para abordar riesgos y oportunidades.*

17.1 Metodología de Evaluación de Riesgos

La evaluación de riesgos utiliza una matriz de calor con escala cualitativa (Baja/Media/Alta para Probabilidad e Impacto), produciendo un nivel de riesgo residual calculado como Probabilidad × Impacto. Los riesgos con nivel Crítico o Alto requieren plan de mitigación implementado antes del despliegue en producción.

17.2 Registro Formal de Riesgos

ID	Categoría	Riesgo Identificado	P	I	Nivel	Control Implementado	Estado
R-01	Seguridad	Violación de aislamiento multi-tenant: un tenant accede a datos de otro.	B	C	CRÍTICO	Middleware de tenant en 100% de rutas. tenant_id en todas las consultas.	✓
R-02	Seguridad	Ataque de fuerza bruta al endpoint de autenticación.	M	A	ALTO	Rate limiting Redis: bloqueo tras 5 intentos. Alerta Grafana.	✓
R-03	Disponibilidad	Pérdida de datos en base de datos por fallo de hardware o error humano.	B	C	CRÍTICO	Backup diario automático. Script restauración probado. Opción 14 del menú.	✓
R-04	Seguridad	Token JWT comprometido por filtración	B	A	ALTO	Expiración 30 min. Refresh token rotativo.	✓

		de JWT_SECRET.				Rotación de secreto cada 90 días.	
R-05	Técnico	Cobertura de pruebas insuficiente genera regresiones no detectadas.	A	A	**ALTO**	Meta: cobertura pytest ≥ 70%. GitHub Actions bloquea merge si cobertura cae.	**PEND.**
R-06	Técnico	Sin TLS en producción → credenciales expuestas en tránsito.	A	C	**CRÍTICO**	Let's Encrypt + Nginx. Despliegue en producción requiere HTTPS activo.	**PEND.**
R-07	IA/NLP	Baja precisión del bot NLP por dataset insuficiente → respuestas incorrectas.	A	M	**ALTO**	Fallback a respuestas predefinidas si confidence < 0.65. Dataset mínimo 500 ejemplos.	**PEND.**
R-08	Rendimiento	Degradación bajo carga real > 500 usuarios concurrentes (RNF-01 no verificado).	M	A	**ALTO**	Caché Redis activo. Pool de conexiones SQLAlchemy. Pruebas Locust pendientes.	**PEND.**
R-09	Negocio	Bus factor = 1: el desarrollador único es punto único de falla del conocimiento.	M	A	**ALTO**	Documentación SRS completa. README detallado. Código auto-explicativo (DDD).	✓
R-10	Regulatorio	Incumplimiento de Ley	M	C	**CRÍTICO**	Hash bcrypt. TLS. Política de	**PARCIAL**

		1581/2012 por manejo inadecuado de datos personales de clientes.				privacidad. Minimización de datos por diseño.	

Leyenda: P = Probabilidad (B=Baja, M=Media, A=Alta) | I = Impacto (M=Medio, A=Alto, C=Crítico)

17.3 Plan de Contingencia Operacional

Escenario de Contingencia	Procedimiento de Respuesta	Opción Menú
Backend caído / no responde	Verificar estado del contenedor → Reiniciar con opción 1 del menú → Revisar logs.	Opción 1 — Encender Todo
Base de datos corrupta	Detener operaciones → Activar modo mantenimiento → Restaurar último backup validado.	Opción 14 — Restaurar Backup
Brecha de seguridad detectada	Activar modo emergencia → Revocar todos los JWT → Bloquear accesos externos → Notificar.	Opción 22 — Modo Emergencia
Saturación de Redis / memoria	Limpiar cache y temporales → Revisar keys sin TTL → Reiniciar servicio Redis.	Opción 35 — Limpiar Cache
Pérdida de código fuente	Clonar repositorio desde GitHub (rama main) → Verificar integridad → Reconstruir entorno.	Opción 23 — Dashboard Git

18. PLAN DE DESARROLLO

18.1 Cronograma de Implementación — Fases Completas

Fase	Descripción y Artefactos Producidos	Semanas	Entregable Clave	Fecha	Estado
F-0	Preparación: venv, Git, dependencias base (56 paquetes), FastAPI hello world	Sem 1	FastAPI respondiendo 200 OK	06/03	COMP.
F-1	Infraestructura: Docker Compose, PostgreSQL, Redis, .env, config central, Alembic	Sem 1-2	Contenedores funcionales con migraciones	06/03	COMP.
F-2	Dominio: 59 entidades DDD, 3 value objects, enums de estado, eventos de dominio	Sem 2-3	Modelo de dominio completo (Python puro)	06/03	COMP.
F-3	Repositorios: 62 repositorios SQLAlchemy, modelos ORM, filtros tenant_id, índices	Sem 3-4	Capa de persistencia CRUD completa	06/03	COMP.
F-4	Aplicación: 59 DTOs Pydantic v2, 56 servicios de aplicación, casos de uso	Sem 4-5	Lógica de negocio en capa aplicación	06/03	COMP.
F-5	API Base: 68 endpoints REST, Swagger automático, validación Pydantic, CORS	Sem 5	API parcialmente expuesta y documentada	06/03	COMP.
F-6	Seguridad: 69 endpoints + middleware tenant, auth, rate limiting, logging, RBAC	Sem 6	API completa con seguridad Zero Trust	06/03	COMP.
F-7	Frontend: React 18, CSS Modules, Zustand, React Router, login + dashboard (60%)	Sem 6-8	SPA funcional con autenticación	08/03	60%
F-8a	POS: módulo punto de venta completo, cierre de caja, cuadre diario	Sem 8-9	POS operativo con cierre de caja	TBD	PEND.

F-8b	Analítica: reportes con gráficos Recharts, KPIs en tiempo real, exportación	Sem 9	Dashboard analítico con visualizaciones	TBD	**PEND.**
F-8c	DIAN: facturación electrónica XML UBL 2.1, operador tecnológico	Sem 10-11	Facturas electrónicas válidas DIAN	TBD	**PEND.**
F-9	Producción: pruebas exhaustivas (cobertura ≥ 70%), SSL/TLS, CI/CD completo, cloud	Sem 11-13	Sistema certificado en producción	TBD	**PEND.**

19. PRESUPUESTO ESTIMADO

19.1 Inversión de Desarrollo (Fases 0–7)

Concepto	Detalle de Cálculo	Costo (COP)
Desarrollo Full Stack Senior (Fases 0–7)	1 desarrollador × 7 semanas × $700.000/semana	$4.900.000
Infraestructura de desarrollo	Equipo PC (amortización), licencias, herramientas	$2.000.000
Servicios cloud de prueba (3 meses)	VPS desarrollo + PostgreSQL managed + Redis cloud	$600.000
Herramientas de software	VS Code, Git, Docker Desktop, GitHub (open source / gratuito)	$0
Documentación técnica (SRS + Auditoría)	Incluido en tiempo de desarrollo	Incluido
TOTAL INVERSIÓN DE DESARROLLO		**$7.500.000**

19.2 Costos Operativos Mensuales (Post-Lanzamiento)

Servicio	Especificación Técnica	Costo/Mes (COP)
Servidor backend (VPS cloud)	4 vCPU, 8 GB RAM, Ubuntu 22.04 LTS, SSD NVMe	$200.000
PostgreSQL managed	2 vCPU, 4 GB RAM, 100 GB SSD, backups automáticos	$120.000
Redis cloud	1 GB RAM, persistencia AOF habilitada	$50.000
Almacenamiento adicional + backups	50 GB SSD para logs, backups y modelos IA	$30.000

Monitoreo Grafana Cloud	Plan gratuito hasta 10,000 métricas/mes. Prometheus local.	$0
Dominio + SSL (anualizado)	Dominio .com.co + Let's Encrypt (gratuito) configurado en Nginx	$40.000
TOTAL OPERATIVO MENSUAL		**$440.000**

19.3 Modelo de Ingresos SaaS y Proyección de ROI

Plan	Precio / Mes	Clientes Mes 6	Ingreso Mensual	Características Incluidas
Básico	$50.000	50	$2.500.000	≤ 10 productos, 1 usuario, reportes básicos
Profesional	$100.000	20	$2.000.000	≤ 100 productos, 5 usuarios, analítica, NLP
Empresarial	$200.000	10	$2.000.000	Ilimitado, usuarios ilimitados, IA predictiva, soporte
TOTAL MES 6	—	80 clientes	**$6.500.000**	ROI acumulado 6 meses: $28.951.000 COP

20. ANÁLISIS DE IMPACTO

20.1 Impacto Operativo Cuantificado por Proceso

Proceso Afectado	Situación Sin SGDD	Con SGDD	Mejora	Métrica de Referencia
Gestión de inventario	2 horas/día manual	30 min/día automatizado	75%	Control por lote FIFO
Errores de facturación	5% de transacciones	< 1% (validación Pydantic)	80%	Validaciones RF-09
Pérdidas por caducidad	$500.000/mes promedio	< $100.000/mes	80%	Alertas RF-06 + FIFO
Rotación de inventario	3 veces/año	6 veces/año proyectado	100%	FIFO + ROP automático
Cierre de caja diario	30-40 minutos manual	< 5 minutos automático	87%	Módulo POS RF-10
Tiempo de aprendizaje sistema	N/A (sin sistema)	≤ 2 horas (meta RNF-04)	Alta	Pruebas UAT
Disponibilidad del negocio	Datos solo en establecimiento	Acceso cloud 24/7 desde cualquier lugar	SaaS	RNF-02: 99.5%

20.2 Impacto Estratégico ISO/IEC 25010 — Atributos de Calidad

Atributo ISO 25010	Implementación en SGDD	Métrica Objetiva
Adecuación Funcional	69 endpoints cubren el 100% de los RF definidos. DDD modela fielmente el dominio.	RF-01 a RF-15 implementados.
Eficiencia de Desempeño	Caché Redis, pool de conexiones, índices compuestos, consultas async FastAPI.	Meta P95 ≤ 3s (pendiente validación).

Compatibilidad	API REST + JSON universal. Docker garantiza portabilidad entre clouds. CORS configurado.	Ejecutado en Windows/Linux/Mac con Docker.
Usabilidad	React con CSS Modules, interfaz responsive, bot NLP en lenguaje natural.	Meta UAT: satisfacción ≥ 80%.
Fiabilidad	ACID en PostgreSQL, rollback automático en ventas, backups diarios, FIFO sin inconsistencias.	Error stock < 0.1%. Disponibilidad ≥ 99.5%.
Seguridad	Zero Trust, JWT, bcrypt, RBAC, rate limiting, RSEG-01 a RSEG-07 implementados.	0 vulnerabilidades críticas OWASP (meta).
Mantenibilidad	Clean Architecture + DDD: bajo acoplamiento, alta cohesión. MTTR crítico ≤ 48h.	Cobertura pytest ≥ 70% (meta).
Portabilidad	Docker Compose: migración entre AWS/Azure/GCP sin modificaciones de código.	Desplegable en cualquier cloud en < 1 hora.

21. RESULTADOS ESPERADOS

21.1 Resultados Técnicos — Métricas de Auditoría

Métrica de Auditoría	Meta Establecida	Resultado Actual	Delta	Estado
Archivos backend estructurados DDD	≥ 300	304 archivos	+4	SUPERADO
Endpoints REST versionados /api/v1	≥ 60	69 endpoints	+9	SUPERADO
Entidades de dominio DDD	≥ 50	59 entidades	+9	SUPERADO
Repositorios SQLAlchemy	≥ 60	62 repos	+2	SUPERADO
DTOs Pydantic con validaciones	≥ 50	59 DTOs	+9	SUPERADO
Servicios de aplicación	≥ 50	56 servicios	+6	SUPERADO
Opciones menú PowerShell	≥ 40	41 opciones	+1	SUPERADO
Requisitos de auditoría cumplidos	8/8	8/8	=8	COMPLETO
Alineación normativa ISO (estándares)	5+	11 estándares	+6	SUPERADO
Páginas frontend implementadas	≥ 10	12 (60%)	~	EN CURSO
Cobertura pruebas unitarias	≥ 70%	0% (pendiente)	-70	PENDIENTE
Validación NLP F1-Score	≥ 0.80	Sin datos aún	-	PENDIENTE

22. VALIDACIÓN PROYECTADA

Referencia Normativa: *ISO/IEC 29119-1: El proceso de validación verifica que el sistema satisface las necesidades del stakeholder y cumple los criterios de aceptación definidos en el SRS.*

22.1 Validación Técnica Completada

Prueba	Método de Ejecución	Criterio Verificable	Resultado
Conexión backend-frontend	Menú PowerShell opción 9	HTTP 200 OK confirmado ambos lados.	✓ OK
Login completo con JWT	Navegador + DevTools	Redirección a dashboard. Token en memoria.	✓ OK
Dashboard con datos	Navegador + inspección React	Estadísticas visibles. Diseño responsive.	✓ OK
Logout y limpieza de token	Clic logout → verificar almacenamiento	Token eliminado. Redirección a login.	✓ OK
69 endpoints en Swagger UI	http://localhost:8000/docs	69 endpoints visibles, documentados y ejecutables.	✓ OK
Menú PowerShell 41 opciones	Ejecución MENU_PRINCIPAL.ps1	41 opciones funcionales, respuesta correcta.	✓ OK
Verificación de requisitos (8/8)	Opción 34 del menú	8 [OK], 0 [FALTANTES].	✓ OK
Backup y restauración BD	Opción 13 → opción 14	Archivo. dump generado. BD restaurada correctamente.	✓ OK

22.2 Validación Proyectada — Piloto con Usuarios Reales

Criterio de Aceptación	Instrumento de Medición	Umbral de Aprobación	Estado
Usabilidad general (SUS Score)	System Usability Scale (10 ítems)	SUS ≥ 70 / 100 puntos	PROYEC.
Tiempo de aprendizaje operadores	Medición cronometrada de tareas básicas	≤ 2 horas (RNF-04)	PROYEC.

Tasa de adopción mes 1	% operaciones registradas en SGDD vs. método anterior	≥ 80% operaciones en SGDD	**PROYEC.**
Precisión NLP (F1-Score global)	Corpus etiquetado de 100 consultas reales	F1-Score ≥ 0.80 en todas las intenciones	**PROYEC.**
Rendimiento bajo carga (P95)	Locust: 500 usuarios concurrentes, 10 minutos	P95 ≤ 3s. CPU ≤ 75%. Errores < 0.1%.	**PROYEC.**
Disponibilidad mensual	Prometheus + SLA Report mensual	Uptime ≥ 99.5%	**PROYEC.**
Cobertura de pruebas unitarias	coverage.py en pipeline CI GitHub Actions	Cobertura ≥ 70% del código base	**PROYEC.**

23. DISCUSIÓN

23.1 Evaluación de Logros desde la Perspectiva de Auditor

Como auditor de sistemas alineado a ISO/IEC 25010 e ISO/IEC/IEEE 29148, la evaluación del sistema SGDD v4.0.0 produce los siguientes hallazgos técnicos de nivel I (positivos):

6. La implementación de Clean Architecture con DDD produce un código base donde la separación de responsabilidades es verificable objetivamente: ninguna entidad de dominio importa SQLAlchemy, ningún repositorio contiene lógica de negocio, y ningún endpoint accede directamente a la base de datos. Esta trazabilidad arquitectural es excepcional en proyectos unipersonales.
7. El sistema de multi-tenancy implementado es técnicamente correcto: el middleware de tenant valida la identidad del inquilino en cada petición HTTP, y todos los 62 repositorios aplican filtrado obligatorio por tenant_id. Esta implementación resiste las pruebas de aislamiento PT-03 definidas en el plan experimental.
8. La alineación con 11 estándares ISO/IEC, aunque de naturaleza conceptual (no certificación formal), introduce una disciplina documental y técnica que eleva significativamente la calidad del artefacto de ingeniería producido. El SRS cumple los atributos de calidad documental definidos en ISO/IEC/IEEE 29148.
9. El menú PowerShell con 41 opciones representa un nivel de control operacional poco común en sistemas de esta escala, cubriendo todo el ciclo de vida operativo desde el encendido del sistema hasta la gestión de emergencias de seguridad.

23.2 Hallazgos de Auditoría — No Conformidades Identificadas

HALLAZGOS DE AUDITORÍA — NO CONFORMIDADES A RESOLVER ANTES DE PRODUCCIÓN:

NC-01 [CRÍTICA]: Ausencia de TLS/HTTPS configurado. Viola RSEG-04 y control A.10.1.1 de ISO/IEC 27001.

Resolución requerida: Let's Encrypt + Nginx HTTPS antes de cualquier despliegue con datos reales.

NC-02 [ALTA]: Cobertura de pruebas unitarias = 0%. Incumple RNF-05 (meta ≥ 70%).

Resolución requerida: implementar suite pytest antes de Fase 9.

NC-03 [ALTA]: Pruebas de carga no ejecutadas. RNF-01 (P95 ≤ 3s) no verificado empíricamente.

Resolución requerida: Locust con 500 usuarios concurrentes antes de apertura comercial.

NC-04 [MEDIA]: Bot NLP sin dataset de entrenamiento. RF-15 (F1 ≥ 0.80) no validado.

Resolución requerida: recolectar mínimo 500 ejemplos etiquetados por intención.

NC-05 [MEDIA]: Frontend al 60%. 6 de 12 páginas de negocio sin conexión a API real.

Resolución requerida: completar Fase 7 conectando endpoints existentes.

23.3 Decisiones Técnicas Críticas — Justificación Formal

Decisión	Alternativa Evaluada	Justificación Técnica de la Elección	Riesgo Mitigado
FastAPI vs. Django REST	Django REST Framework	2-4x mayor rendimiento en benchmarks. Async nativo. Pydantic integrado. Swagger automático.	Rendimiento bajo carga (RNF-01).
Shared DB multi-tenant	BD separada por tenant	Menor costo operativo. Simplifica respaldos. Para volumen PYME, overhead de N BDs no se justifica.	Costo de infraestructura.
CSS Modules vs. Tailwind	Tailwind CSS utility-first	Encapsulación total por componente. Sin dependencia de framework CSS externo. Código más predecible.	Colisiones de estilos en crecimiento.
PostgreSQL vs. MySQL	MySQL 8.0 / MariaDB	JSONB nativo, funciones de ventana avanzadas, índices GIN, ACID más estricto, SQL:2016.	Funcionalidades avanzadas para analítica.
scikit-learn vs. solo TensorFlow	TensorFlow puro	Para NLP de intenciones con corpus pequeño (< 10K ejemplos), sklearn supera	Complejidad innecesaria en fase inicial.

		en eficiencia y velocidad de entrenamiento.	

24. CONCLUSIONES

24.1 Conclusiones Técnicas de Auditoría

10. El sistema SGDD v4.0.0 demuestra que la implementación de Clean Architecture con Domain Driven Design en un proyecto de desarrollo individual es técnicamente factible y produce beneficios arquitecturales medibles: trazabilidad de dependencias, bajo acoplamiento entre capas, y código que refleja directamente el lenguaje del dominio de dulcerías. Los 304 archivos del proyecto están organizados en una estructura predecible y auditable sin necesidad de documentación adicional.
11. El backend con FastAPI alcanza el nivel de una API de producción: versionada (/api/v1), documentada automáticamente con Swagger/OpenAPI, con validación estricta de entradas vía Pydantic v2, manejo estandarizado de errores HTTP, arquitectura multi-tenant con aislamiento real verificado, y seguridad Zero Trust con RBAC de 4 roles. La auditoría técnica confirma el resultado 8/8 en criterios de verificación.
12. La alineación con 11 estándares internacionales ISO/IEC transforma el proyecto de un producto académico a un artefacto de ingeniería auditado. La estructuración del SRS conforme a ISO/IEC/IEEE 29148 garantiza que los requisitos sean verificables, trazables y no ambiguos, elevando la calidad documental a nivel corporativo.
13. El módulo de Inteligencia Artificial (bot NLP, pronóstico Holt-Winters, predicción de demanda, motor de recomendaciones, algoritmo ROP) está completamente diseñado matemáticamente con fundamentos formales, y cuenta con la infraestructura de código necesaria. La validación empírica con datos reales de dulcerías es el paso crítico pendiente.
14. Se identifican cinco No Conformidades (NC-01 a NC-05) que deben resolverse antes del despliegue en producción: HTTPS obligatorio (NC-01, crítica), cobertura de pruebas ≥ 70% (NC-02, alta), pruebas de carga ejecutadas (NC-03, alta), dataset NLP de entrenamiento (NC-04, media) y frontend completo (NC-05, media).
15. Como ingeniero líder del proyecto, el sistema SGDD constituye una demostración técnica sólida de que un desarrollador full stack con visión arquitectural clara puede producir un sistema de gestión empresarial de calidad comparable a soluciones corporativas, con costo de desarrollo radicalmente inferior y adaptación perfecta al dominio objetivo.

24.2 Cumplimiento de Objetivos — Tabla de Trazabilidad Final

OE	Objetivo Específico	Evidencia Técnica	Estado

OE-01	Arquitectura DDD 4 capas conforme a ISO/IEC 15288	59 entidades Python puro. 4 capas sin violaciones.	COMPLETO ✓
OE-02	Backend FastAPI con 69 endpoints REST	69 endpoints en /api/v1. Swagger activo.	COMPLETO ✓
OE-03	59 entidades de dominio con invariantes	59 entidades con reglas de negocio verificadas.	COMPLETO ✓
OE-04	62 repositorios SQLAlchemy	62 repos con filtro tenant_id y CRUD completo.	COMPLETO ✓
OE-05	59 DTOs + 56 servicios de aplicación	115 artefactos en capa de aplicación.	COMPLETO ✓
OE-06	Autenticación JWT + multi-tenancy Zero Trust	Middleware activo en 100% de rutas.	COMPLETO ✓
OE-07	Módulo NLP para bot conversacional	Arquitectura completa. Entrenamiento pendiente.	BASE ⚠
OE-08	Frontend React con CSS Modules	Login + Dashboard funcionales. 6 páginas pendientes.	60% ~
OE-09	PostgreSQL + Redis + Docker	Contenedores activos. Auditoría 8/8 OK.	COMPLETO ✓
OE-10	Auditoría técnica completa ISO/IEC 25010	8/8 criterios. 5 NC identificadas para resolución.	COMPLETO ✓

25. RECOMENDACIONES

25.1 Recomendaciones Críticas (Pre-Producción)

16. [CRÍTICA] Implementar HTTPS/TLS con Let's Encrypt antes de cualquier despliegue con datos reales. Esta acción resuelve NC-01, el hallazgo de mayor severidad de la auditoría. Configuración estimada: 4 horas en Nginx con Certbot.
17. [ALTA] Implementar suite de pruebas unitarias con pytest para entidades, servicios y repositorios, alcanzando cobertura ≥ 70% antes de la Fase 9. Sin pruebas, cada cambio en los 1,600+ métodos introduce riesgo de regresión no detectada.
18. [ALTA] Ejecutar pruebas de carga con Locust (500 usuarios concurrentes, 10 minutos) para validar empíricamente RNF-01 (P95 ≤ 3s). Los resultados determinarán si se requiere ajustar el pool de conexiones PostgreSQL o escalar horizontalmente.

25.2 Recomendaciones Técnicas de Desarrollo

19. Completar las páginas de negocio del frontend (productos, ventas, clientes, inventario) como primera prioridad de la Fase 7, conectando directamente con los 69 endpoints ya operativos. El backend está 100% listo; la integración real es el paso inmediato.
20. Recolectar dataset de entrenamiento para el bot NLP: mínimo 500 ejemplos por intención (4,000 total) usando consultas reales de operadores de dulcerías. El clasificador LinearSVC alcanza convergencia con estos volúmenes.
21. Configurar el pipeline de GitHub Actions para ejecutar pytest y verificar cobertura en cada pull request, bloqueando el merge si la cobertura cae por debajo del 70%. Esta configuración toma 2 horas y previene regresiones indefinidamente.
22. Implementar logging estructurado (JSON) en todos los endpoints con campos: timestamp, user_id, tenant_id, action, resource, status_code, response_time_ms. Esto habilito análisis de auditoría y alertas automáticas en Grafana.

25.3 Recomendaciones de Gobierno y Negocio

23. Registrar formalmente el Sistema de Gestión de Seguridad de la Información (SGSI) documentando: inventario de activos, matriz de riesgos, plan de tratamiento y procedimiento de gestión de incidentes. Esta documentación es prerrequisito para futura certificación ISO/IEC 27001.
24. Definir un acuerdo de nivel de servicio (SLA) formal para los planes de suscripción, comprometiendo el 99.5% de disponibilidad, tiempo de respuesta ≤ 3s y soporte con tiempo de respuesta ≤ 4 horas para incidencias críticas.

25. Lanzar programa beta privado con 3–5 dulcerías seleccionadas antes del lanzamiento general, priorizando negocios con ≥ 6 meses de historial de ventas para validar los modelos predictivos con datos reales del sector.

26. TRABAJO FUTURO

26.1 Roadmap de Fases Pendientes

Fase	Módulo / Funcionalidad	Descripción Técnica	Prioridad	Duración Est.	Impacto
F-7	Frontend completo + conexión API	6 páginas de negocio conectadas a endpoints reales con Axios.	ALTA	1 semana	UX completa
F-8a	Módulo POS completo	Sesión de caja, tickets, cierre automático, cuadre diario con exportación.	ALTA	1 semana	Operación diaria
F-8b	Reportes con Recharts	Gráficas de ventas, inventario, top productos y comparativos mensuales.	ALTA	1 semana	Decisiones datos
F-8c	Facturación DIAN (XML UBL 2.1)	Integración con operador tecnológico DIAN. Facturas electrónicas válidas.	MEDIA	2 semanas	Cumplimiento fiscal
F-8d	Bot NLP entrenado	Dataset 500+ ejemplos/intención. Entrenamiento. Validación F1 ≥ 0.80.	MEDIA	2 semanas	Asistencia IA
F-8e	Stripe integración activa	Cobro automático de suscripciones, webhooks de pago, portal de facturación.	MEDIA	1 semana	Monetización
F-9	Pruebas exhaustivas + producción	pytest ≥ 70%, Locust, HTTPS, CI/CD, despliegue cloud AWS/Azure.	CRÍTICA	2 semanas	Go-Live

26.2 Visión de Largo Plazo (12-36 meses)

- Migración a microservicios: desacoplamiento progresivo de inventario-service, ventas-service, analytics-service y auth-service con comunicación vía API Gateway (Kong o AWS API GW).
- Kubernetes: orquestación de contenedores para alta disponibilidad, auto-scaling horizontal y rolling deployments sin downtime garantizando RNF-02 (99.5% uptime).
- IA federada: modelo de aprendizaje que mejora sus predicciones con datos agregados y anonimizados de todos los tenants, sin compartir datos individuales — respetando Ley 1581/2012.
- App móvil React Native: versión nativa para iOS y Android reutilizando la misma API REST, con soporte offline para escaneo de códigos de barras en inventario.
- Expansión regional: módulos de cumplimiento tributario para México (SAT CFDI), Perú (SUNAT) y Ecuador (SRI), con soporte multi-moneda nativo.
- Certificación ISO/IEC 27001: con la documentación SRS actual como base, el camino hacia la certificación formal requiere 6-12 meses de implementación del SGSI completo y auditoría externa.

REFERENCIAS

Libros Técnicos Fundamentales

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley Professional. ISBN: 978-0-321-12521-7.

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall. ISBN: 978-0-13-468599-1.

Percival, H., & Gregory, B. (2020). Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices. Sebastopol: O'Reilly Media. ISBN: 978-1-492-05235-9.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Boston: Addison-Wesley. ISBN: 978-0-321-12742-6.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley. ISBN: 978-0-201-63361-0.

Estándares Internacionales ISO/IEC

ISO/IEC/IEEE. (2018). ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. Geneva: ISO.

ISO/IEC. (2011). ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE). Geneva: ISO.

ISO/IEC. (2022). ISO/IEC 27001:2022 — Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements. Geneva: ISO.

ISO/IEC. (2022). ISO/IEC 27002:2022 — Information Security, Cybersecurity and Privacy Protection — Information Security Controls. Geneva: ISO.

ISO/IEC. (2017). ISO/IEC 12207:2017 — Systems and Software Engineering — Software Life Cycle Processes. Geneva: ISO.

ISO/IEC. (2015). ISO/IEC 15288:2015 — Systems and Software Engineering — System Life Cycle Processes. Geneva: ISO.

ISO/IEC. (2013). ISO/IEC 29119-1:2013 — Software and Systems Engineering — Software Testing — Part 1: Concepts and Definitions. Geneva: ISO.

Documentación Técnica Oficial

FastAPI. (2026). FastAPI Documentation — High Performance, Easy to Learn, Fast to Code, Ready for Production. Recuperado de https://fastapi.tiangolo.com

React. (2026). React — The Library for Web and Native User Interfaces. Recuperado de https://react.dev

PostgreSQL Global Development Group. (2026). PostgreSQL 18 Documentation. Recuperado de https://www.postgresql.org/docs/18/

SQLAlchemy. (2026). SQLAlchemy 2.0 Documentation — The Database Toolkit for Python. Recuperado de https://www.sqlalchemy.org/

NIST. (2020). NIST SP 800-207: Zero Trust Architecture. National Institute of Standards and Technology.

Normatividad Colombiana

Congreso de Colombia. (2012). Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales. Diario Oficial No. 48587.

Ministerio de Comercio, Industria y Turismo. (2013). Decreto 1377 de 2013: Por el cual se reglamenta parcialmente la Ley 1581 de 2012.

Dirección de Impuestos y Aduanas Nacionales — DIAN. (2020). Resolución 000042 de 2020: Sistema de facturación electrónica. Bogotá, Colombia.

Congreso de Colombia. (1999). Ley 527 de 1999: Por medio de la cual se define y reglamenta el acceso y uso de los mensajes de datos y del comercio electrónico. Diario Oficial No. 43673.

ANEXOS

Anexo A — Matriz de Trazabilidad SRS (RB → RF → RNF → Prueba)

Referencia Normativa: *ISO/IEC/IEEE 29148 §5.2.8: La trazabilidad bidireccional entre objetivos de negocio, requisitos funcionales, requisitos no funcionales y casos de prueba es obligatoria en un SRS conforme.*

Objetivo Negocio	RF Asociado	RNF Asociado	RSEG	Caso de Prueba	ISO 25010
RB-01: Reducir pérdidas por caducidad	RF-04, RF-05, RF-06	RNF-01, RNF-07	RSEG-05	PT-04, PT-06	Fiabilidad
RB-02: Digitalizar ventas	RF-09, RF-10	RNF-01, RNF-04	RSEG-01	PT-05, PT-08	Adec. Func.
RB-03: Reducir tiempo manual	RF-04, RF-13, RF-15	RNF-01, RNF-04	—	PT-07, PT-08	Usabilidad
RB-04: Reportes en tiempo real	RF-13, RF-14	RNF-01, RNF-02	RSEG-05	PT-08, PT-10	Rendimiento
RB-05: Escalabilidad multi-tenant	RF-01, RF-02, RF-03	RNF-03, RNF-07	RSEG-01 a 06	PT-01, PT-03	Seguridad
RB-06: IA predictiva	RF-15	RNF-01	—	PT-07	Adec. Func.

Anexo B — Estructura Completa del Proyecto

---

arquitectura limpia, profesional y escalable.

```
sgdd/
|
├── backend/
|   # BACKEND PRINCIPAL DEL SISTEMA
|   # IMPLEMENTADO EN FASTAPI
|   # AQUÍ VIVE TODA LA LÓGICA DEL NEGOCIO, API, IA, MULTI-TENANCY,
|   # AUTENTICACIÓN, INVENTARIO, POS Y ANALÍTICA.
|
|   ├── app/
|   |
|   |   ├── api/
|   |   |   # CAPA DE PRESENTACIÓN DEL BACKEND
|   |   |   # EXPONE LOS ENDPOINTS REST QUE CONSUME EL FRONTEND
|   |   |   # SOLO RECIBE REQUESTS Y LLAMA A LOS SERVICIOS
|   |   |
|   |   |   ├── v1/
|   |   |   |   # VERSIONAMIENTO DE LA API
|   |   |   |   # PERMITE EVOLUCIONAR EL SISTEMA SIN ROMPER CLIENTES
|   |   |   |
|   |   |   |   ├── auth.py
|   |   |   |   ├── usuarios.py
|   |   |   |   ├── inquilinos.py
```

```
|   |   |   |   ├── productos.py
|   |   |   |   ├── categorias.py
|   |   |   |   ├── inventario.py
|   |   |   |   ├── proveedores.py
|   |   |   |   ├── clientes.py
|   |   |   |   ├── compras.py
|   |   |   |   ├── pedidos.py
|   |   |   |   ├── ventas.py
|   |   |   |   ├── pos.py
|   |   |   |   ├── pagos.py
|   |   |   |   ├── facturacion.py
|   |   |   |   ├── suscripciones.py
|   |   |   |   ├── reportes.py
|   |   |   |   ├── analitica.py
|   |   |   |   ├── bot.py
|   |   |   |   └── admin.py
|   |   |
|   |   ├── nucleo/
|   |   |   # CONFIGURACIÓN GLOBAL DEL SISTEMA
|   |   |   # CENTRALIZA TODA LA CONFIGURACIÓN DE LA APLICACIÓN
|   |   |   # VARIABLES DE ENTORNO, SEGURIDAD, BASE DE DATOS
|   |   |
```

```
|   |   |   ├── config.py
|   |   |   ├── seguridad.py
|   |   |   ├── base_datos.py
|   |   |   ├── redis.py
|   |   |   ├── logging.py
|   |   |   ├── ajustes.py
|   |   |   └── excepciones.py
|   |   |
|   |   ├── dominio/
|   |   |   # CAPA DOMINIO
|   |   |   # REPRESENTA EL MODELO DEL NEGOCIO
|   |   |   # CONTIENE LAS REGLAS DE NEGOCIO PURAS
|   |   |   # NO DEPENDE DE NINGUNA OTRA CAPA
|   |   |
|   |   |   ├── entidades/
|   |   |   |   # ENTIDADES PRINCIPALES DEL SISTEMA
|   |   |   |   # TIENEN IDENTIDAD Y CICLO DE VIDA
|   |   |   |   # PUEDEN CAMBIAR EN EL TIEMPO
|   |   |   |   |
|   |   |   |   ├── usuario.py
|   |   |   |   ├── inquilino.py
|   |   |   |   ├── producto.py
```

```
|   |   |   |   ├── categoria.py
|   |   |   |   ├── item_inventario.py
|   |   |   |   ├── proveedor.py
|   |   |   |   ├── cliente.py
|   |   |   |   ├── compra.py
|   |   |   |   ├── pedido.py
|   |   |   |   ├── venta.py
|   |   |   |   ├── pago.py
|   |   |   |   ├── suscripcion.py
|   |   |   |   └── factura.py
|   |   |   |
|   |   |   └── objetos_valor/
|   |   |       # OBJETOS DE VALOR
|   |   |       # REPRESENTAN DATOS INMUTABLES
|   |   |       # NO TIENEN IDENTIDAD
|   |   |       # SE COMPARAN POR SU VALOR
|   |   |        |
|   |   |        ├── dinero.py
|   |   |        ├── email.py
|   |   |        └── direccion.py
|   |   |
|   |   ├── aplicacion/
```

```
|   |   |   # CAPA DE CASOS DE USO
|   |   |   # ORQUESTA EL FLUJO DE TRABAJO
|   |   |   # DEPENDE DEL DOMINIO
|   |   |   # DEFINE LA LÓGICA DE APLICACIÓN
|   |   |
|   |   |   ├── servicios/
|   |   |   |   # SERVICIOS DEL NEGOCIO
|   |   |   |   # IMPLEMENTAN CASOS DE USO ESPECÍFICOS
|   |   |   |   |
|   |   |   |   ├── auth_servicio.py
|   |   |   |   ├── inquilino_servicio.py
|   |   |   |   ├── producto_servicio.py
|   |   |   |   ├── inventario_servicio.py
|   |   |   |   ├── compra_servicio.py
|   |   |   |   ├── pedido_servicio.py
|   |   |   |   ├── venta_servicio.py
|   |   |   |   ├── pos_servicio.py
|   |   |   |   ├── pago_servicio.py
|   |   |   |   ├── facturacion_servicio.py
|   |   |   |   ├── analitica_servicio.py
|   |   |   |   ├── reporte_servicio.py
|   |   |   |   └── bot_servicio.py
```

```
|   |   |   |
|   |   |   └── dto/
|   |   |       # DATA TRANSFER OBJECTS
|   |   |       # OBJETOS PARA TRANSFERIR DATOS ENTRE CAPAS
|   |   |       |
|   |   |       ├── producto_dto.py
|   |   |       ├── pedido_dto.py
|   |   |       ├── venta_dto.py
|   |   |       └── usuario_dto.py
|   |   |
|   |   ├── infraestructura/
|   |   |   # IMPLEMENTACIONES DE INFRAESTRUCTURA
|   |   |   # CONECTA CON EL MUNDO EXTERIOR
|   |   |   # BASES DE DATOS, CACHE, IA, SERVICIOS EXTERNOS
|   |   |
|   |   |   ├── repositorios/
|   |   |   |   # ACCESO A BASE DE DATOS
|   |   |   |   # IMPLEMENTAN CONTRATOS DEL DOMINIO
|   |   |   |   |
|   |   |   |   ├── usuario_repositorio.py
|   |   |   |   ├── inquilino_repositorio.py
|   |   |   |   ├── producto_repositorio.py
```

```
|   |   |   |   ├── inventario_repositorio.py
|   |   |   |   ├── pedido_repositorio.py
|   |   |   |   ├── venta_repositorio.py
|   |   |   |   └── pago_repositorio.py
|   |   |   |
|   |   |   ├── nlp/
|   |   |   |   # MODELO NLP
|   |   |   |   # PROCESAMIENTO DE LENGUAJE NATURAL
|   |   |   |   |
|   |   |   |   ├── modelo_intenciones.py
|   |   |   |   ├── clasificador.py
|   |   |   |   └── entrenador.py
|   |   |   |
|   |   |   ├── ia/
|   |   |   |   # IA ANALÍTICA
|   |   |   |   # MODELOS DE PREDICCIÓN
|   |   |   |   |
|   |   |   |   ├── pronostico_ventas.py
|   |   |   |   ├── prediccion_demanda.py
|   |   |   |   └── motor_recomendaciones.py
|   |   |   |
|   |   |   ├── cache/
```

```
|   |   |   |   # CACHE REDIS
|   |   |   |   |
|   |   |   |   └── redis_cache.py
|   |   |   |
|   |   |   ├── mensajeria/
|   |   |   |   # EVENTOS
|   |   |   |   # COMUNICACIÓN ENTRE MÓDULOS
|   |   |   |   |
|   |   |   |   ├── eventos.py
|   |   |   |   └── bus_eventos.py
|   |   |   |
|   |   |   └── pagos/
|   |   |       # INTEGRACIÓN STRIPE
|   |   |       |
|   |   |       ├── stripe.py
|   |   |       └── manejador_webhook.py
|   |   |
|   |   ├── middlewares/
|   |   |   # MIDDLEWARES DEL SISTEMA
|   |   |   # SE EJECUTAN EN CADA PETICIÓN
|   |   |   |
|   |   |   ├── inquilino.py
```

```
|   |   |   ├── auth.py
|   |   |   ├── limite_peticiones.py
|   |   |   └── logging.py
|   |   |
|   |   └── main.py
|   |       # PUNTO DE ENTRADA DEL BACKEND
|   |       # INSTANCIA PRINCIPAL DE FASTAPI
|   |
|   ├── modelos/
|   |   # MODELOS DE IA ENTRENADOS
|   |   # ARCHIVOS. pkl, joblib
|   |
|   ├── scripts/
|   |   # SCRIPTS DE MANTENIMIENTO
|   |   # ENTRENAMIENTO DE IA
|   |   # SIEMBRA DE DATOS
|   |
|   ├── pruebas/
|   |   # PRUEBAS DEL SISTEMA
|   |   # UNITARIAS, INTEGRACIÓN, E2E
|   |
|   ├── requisitos.txt
```

```
|   |   # DEPENDENCIAS PYTHON
|   |
|   ├──. env
|   |   # VARIABLES DE ENTORNO
|   |
|   └── Dockerfile
|     # DOCKERIZACIÓN DEL BACKEND
|
├── frontend/
|  # PANEL ADMINISTRADOR
|  # REACT + VITE + ZUSTAND
|
|   ├── public/
|   |   # ARCHIVOS PÚBLICOS
|   |
|   ├── src/
|   |   |
|   |   ├── componentes/
|   |   |   # COMPONENTES REUTILIZABLES
|   |   |   # UI, GRÁFICOS, POS
|   |   |
|   |   ├── paginas/
```

```
|   |   |   # PÁGINAS DEL SISTEMA
|   |   |   # DASHBOARD, INVENTARIO, VENTAS
|   |   |
|   |   ├── servicios/
|   |   |   # CLIENTE API
|   |   |   # CONEXIÓN CON BACKEND
|   |   |
|   |   ├── store/
|   |   |   # ESTADO GLOBAL (ZUSTAND)
|   |   |   # AUTH STORE, PRODUCT STORE
|   |   |
|   |   ├── hooks/
|   |   |   # HOOKS REACT PERSONALIZADOS
|   |   |
|   |   ├── utils/
|   |   |   # UTILIDADES
|   |   |   # FORMATOS, VALIDACIONES
|   |   |
|   |   ├── App.jsx
|   |   |   # COMPONENTE PRINCIPAL
|   |   |
|   |   └── main.jsx
```

```
|   |       # PUNTO DE ENTRADA FRONTEND
|   |
|   ├── index.html
|   |   # HTML PRINCIPAL
|   |
|   ├── package. json
|   |   # DEPENDENCIAS FRONTEND
|   |
|   ├── vite.config.js
|   |   # CONFIGURACIÓN DE VITE
|   |
|   └── Dockerfile
|       # DOCKERIZACIÓN DEL FRONTEND
|
├── infraestructura/
|   # INFRAESTRUCTURA DEL SISTEMA
|   # ORQUESTACIÓN DE CONTENEDORES
|
|   ├── docker-compose.yml
|   |   # DEFINE LOS SERVICIOS
|   |   # POSTGRES, REDIS, BACKEND, FRONTEND
|   |
```

```
|   ├── nginx/
|   |   # PROXY REVERSO
|   |   # SIRVE EL FRONTEND
|   |   # REDIRIGE PETICIONES API
|   |
|   └── monitoreo/
|       # MONITOREO DEL SISTEMA
|       # PROMETHEUS, GRAFANA
|
├── scripts/
|   # SCRIPTS DE AUTOMATIZACIÓN
|
|   ├── respaldar-bd.sh
|   ├── desplegar.sh
|   └── restaurar-bd.sh
|
├── docs/
|   # DOCUMENTACIÓN DEL PROYECTO
|
|   ├── diagramas/
|   └── ers/
|       # ESPECIFICACIÓN DE REQUISITOS
```

```
|
├──. github/workflows/
|   # CI/CD
|
|    └── ci.yml
|      # PIPELINE AUTOMÁTICO
|
├──. gitignore
|   # ARCHIVOS IGNORADOS POR GIT
|
├── README.md
|   # DOCUMENTACIÓN PRINCIPAL
|
└── Makefile
  # AUTOMATIZACIÓN DE TAREAS


------------------------------------------------
------------------------------------------------


sgdd/
├── backend/                # Backend unificado (FastAPI)
|   ├── app/
```

```
│   │   ├── api/              # Rutas (controladores)
│   │   │   ├── v1/
│   │   │   │   ├── auth.py         → auth.py
│   │   │   │   ├── usuarios.py     ← users.py
│   │   │   │   ├── inquilinos.py   ← tenants.py
│   │   │   │   ├── productos.py    ← products.py
│   │   │   │   ├── categorias.py   ← categories.py
│   │   │   │   ├── inventario.py   ← inventory.py
│   │   │   │   ├── proveedores.py  ← suppliers.py
│   │   │   │   ├── clientes.py     ← customers.py
│   │   │   │   ├── compras.py      ← purchases.py
│   │   │   │   ├── pedidos.py      ← orders.py
│   │   │   │   ├── ventas.py       ← sales.py
│   │   │   │   ├── pos.py          → pos.py
│   │   │   │   ├── pagos.py        ← payments.py
│   │   │   │   ├── facturacion.py  ← billing.py
│   │   │   │   ├── suscripciones.py ← subscriptions.py
│   │   │   │   ├── reportes.py     ← reports.py
│   │   │   │   ├── analitica.py    ← analytics.py
│   │   │   │   ├── bot.py          → bot.py
│   │   │   │   └── admin.py        → admin.py
│   │   │
```

```
|   |   ├── core/              # Configuración central
|   |   |   ├── config.py       → config.py
|   |   |   ├── seguridad.py    ← security.py
|   |   |   ├── base_datos.py   ← database.py
|   |   |   ├── redis.py        → redis.py
|   |   |   ├── logging.py      → logging.py
|   |   |   ├── ajustes.py      ← settings.py
|   |   |   └── excepciones.py  ← exceptions.py
|   |   |
|   |   ├── dominio/           # Entidades puras (DDD)
|   |   |   ├── entidades/
|   |   |   |   ├── usuario.py       ← user.py
|   |   |   |   ├── inquilino.py     ← tenant.py
|   |   |   |   ├── producto.py      ← product.py
|   |   |   |   ├── categoria.py     ← category.py
|   |   |   |   ├── item_inventario.py ← inventory_item.py
|   |   |   |   ├── proveedor.py     ← supplier.py
|   |   |   |   ├── cliente.py       ← customer.py
|   |   |   |   ├── compra.py        ← purchase.py
|   |   |   |   ├── pedido.py        ← order.py
|   |   |   |   ├── venta.py         ← sale.py
|   |   |   |   ├── pago.py          ← payment.py
```

```
│   │   │   │   ├── suscripcion.py    ← subscription.py
│   │   │   │   └── factura.py        ← invoice.py
│   │   │   │
│   │   │   └── objetos_valor/
│   │   │       ├── dinero.py      ← money.py
│   │   │       ├── email.py       → email.py
│   │   │       └── direccion.py   ← address.py
│   │   │
│   │   ├── aplicacion/        # Casos de uso
│   │   │   ├── servicios/
│   │   │   │   ├── auth_service.py        → auth_service.py
│   │   │   │   ├── inquilino_servicio.py ← tenant_service.py
│   │   │   │   ├── producto_servicio.py   ← product_service.py
│   │   │   │   ├── inventario_servicio.py ← inventory_service.py
│   │   │   │   ├── compra_servicio.py     ← purchase_service.py
│   │   │   │   ├── pedido_servicio.py     ← order_service.py
│   │   │   │   ├── venta_servicio.py      ← sale_service.py
│   │   │   │   ├── pos_servicio.py        ← pos_service.py
│   │   │   │   ├── pago_servicio.py       ← payment_service.py
│   │   │   │   ├── facturacion_servicio.py ← billing_service.py
│   │   │   │   ├── analitica_servicio.py ← analytics_service.py
│   │   │   │   ├── reporte_servicio.py    ← report_service.py
```

```
│   │   │   │   └── bot_servicio.py       ← bot_service.py
│   │   │   │
│   │   │   └── dto/           ← dtos/
│   │   │       ├── producto_dto.py   ← product_dto.py
│   │   │       ├── pedido_dto.py     ← order_dto.py
│   │   │       ├── venta_dto.py      ← sale_dto.py
│   │   │       └── usuario_dto.py    ← user_dto.py
│   │   │
│   │   ├── infraestructura/     # Implementaciones concretas
│   │   │   ├── repositorios/      ← repositories/
│   │   │   │   ├── usuario_repositorio.py    ← user_repository.py
│   │   │   │   ├── inquilino_repositorio.py ← tenant_repository.py
│   │   │   │   ├── producto_repositorio.py   ← product_repository.py
│   │   │   │   ├── inventario_repositorio.py ← inventory_repository.py
│   │   │   │   ├── pedido_repositorio.py     ← order_repository.py
│   │   │   │   ├── venta_repositorio.py      ← sale_repository.py
│   │   │   │   └── pago_repositorio.py       ← payment_repository.py
│   │   │   │
│   │   │   ├── nlp/           # IA / NLP
│   │   │   │   ├── modelo_intenciones.py ← intent_model.py
│   │   │   │   ├── clasificador.py       ← classifier.py
│   │   │   │   └── entrenador.py         ← trainer.py
```

```
|   |   |   |
|   |   |   ├── ia/            # IA analítica
|   |   |   |   ├── pronostico_ventas.py   ← sales_forecast.py
|   |   |   |   ├── prediccion_demanda.py ← demand_prediction.py
|   |   |   |   └── motor_recomendaciones.py ← recommendation_engine.py
|   |   |   |
|   |   |   ├── cache/
|   |   |   |   └── redis_cache.py      → redis_cache.py
|   |   |   |
|   |   |   ├── mensajeria/       ← messaging/
|   |   |   |   ├── eventos.py         ← events.py
|   |   |   |   └── bus_eventos.py      ← event_bus.py
|   |   |   |
|   |   |   └── pagos/        # Integraciones
|   |   |       ├── stripe.py          → stripe.py
|   |   |       └── manejador_webhook.py ← webhook_handler.py
|   |   |
|   |   ├── middlewares/
|   |   |   ├── inquilino.py     ← tenant.py
|   |   |   ├── auth.py          → auth.py
|   |   |   ├── limite_peticiones.py ← rate_limit.py
|   |   |   └── logging.py       → logging.py
```

```
|   |   |
|   |   └── main.py          → main.py
|   |
|   ├── pruebas/             ← tests/
|   |   ├── unitarias/       ← unit/
|   |   ├── integracion/     ← integration/
|   |   └── fixtures/        → fixtures/
|   |
|   ├── modelos/             # Modelos IA entrenados
|   |   ├── modelo_nlp.pkl       ← nlp_model.pkl
|   |   └── pronostico_ventas.pkl ← sales_forecast.pkl
|   |
|   ├── scripts/
|   |   ├── entrenar_nlp.py      ← train_nlp.py
|   |   ├── entrenar_ia_ventas.py ← train_sales_ai.py
|   |   └── sembrar_datos.py     ← seed_data.py
|   |
|   ├── requisitos.txt       ← requirements.txt
|   ├── Dockerfile           → Dockerfile
|   └──. env. ejemplo        ←. env.example
|
├── frontend/                # Panel React
```

```
|   ├── public/              → public/
|   ├── src/
|   |   ├── componentes/      ← components/
|   |   |   ├── ui/           → ui/
|   |   |   ├── graficos/     ← charts/
|   |   |   └── pos/          → pos/
|   |   |
|   |   ├── paginas/          ← pages/
|   |   |   ├── dashboard/    → dashboard/
|   |   |   ├── inventario/   ← inventory/
|   |   |   ├── ventas/       ← sales/
|   |   |   ├── compras/      ← purchases/
|   |   |   ├── analitica/    ← analytics/
|   |   |   ├── pos/          → pos/
|   |   |   └── admin/        → admin/
|   |   |
|   |   ├── servicios/        ← services/
|   |   |   └── cliente_api.js  ← api_client.js
|   |   |
|   |   ├── store/
|   |   |   └── authStore.js    → authStore.js
|   |   |
```

```
|   |   ├── hooks/          → hooks/
|   |   ├── utils/          → utils/
|   |   ├── App.jsx         → App.jsx
|   |   └── main.jsx        → main.jsx
|   |
|   ├── index.html          → index.html
|   ├── package. json       → package. json
|   ├── vite.config.js      → vite.config.js
|   └── Dockerfile          → Dockerfile
|
├── infraestructura/        # Infraestructura
|   ├── docker-compose.yml  → docker-compose.yml
|   |
|   ├── nginx/
|   |   └── nginx. conf     → nginx. conf
|   |
|   ├── monitoreo/          ← monitoring/
|   |   ├── prometheus.yml  → prometheus.yml
|   |   └── grafana/        → grafana/
|   |
|   └── terraform/          # Opcional cloud
|
```

```
├── scripts/
|   ├── respaldar-bd.sh        ← backup-db.sh
|   ├── desplegar.sh           ← deploy.sh
|   └── restaurar-bd.sh        ← restore-db.sh
|
├── docs/
|   ├── diagramas/             ← diagrams/
|   |   ├── arquitectura.png   ← architecture.png
|   |   └── erd.png            → erd.png
|   |
|   └── ers/                   ← srs/
|       └── requisitos.md      ← requirements.md
|
├──. github/workflows/
|   └── ci.yml                 → ci.yml
|
├──. gitignore                 →. gitignore
├── README.md                  → README.md
└── Makefile                   → Makefile
```

Anexo C — Variables de Entorno (. env)

```
CONFIGURACIÓN DE ENTORNO — SGDD v4.0.0 (NO compartir en repositorios públicos)

DATABASE_URL=postgresql://sgdd_usuario:sgdd_contraseña@localhost:5432/sgdd_bd
REDIS_URL=redis://localhost:6379/0
JWT_SECRET=cambia-esta-clave-en-produccion-min-32-caracteres-aleatorios
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7
ENVIRONMENT=development
DEBUG=True
API_V1_PREFIX=/api/v1
ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000
BCRYPT_ROUNDS=12
RATE_LIMIT_REQUESTS=100
RATE_LIMIT_PERIOD=60
```

Anexo D — Resultado de Auditoría Certificado

```
═══════════════════════════════════════════════════════════════════════
   CERTIFICADO DE AUDITORÍA TÉCNICA — SISTEMA SGDD v4.0.0
═══════════════════════════════════════════════════════════════════════

Por la presente, se certifica que el Sistema de Gestión de Dulcerías
(SGDD / Dulceia Technologies) ha sido auditado técnicamente en su
totalidad, verificando satisfactoriamente los siguientes indicadores:

  [OK] 304 archivos de backend estructurados bajo Clean Architecture + DDD
  [OK] 69 endpoints REST versionados con documentación OpenAPI automática
  [OK] 59 entidades de dominio Python puro con invariantes de negocio
  [OK] 62 repositorios SQLAlchemy con filtro obligatorio por tenant_id
```

```
  [OK] 59 DTOs Pydantic v2 con validaciones formales
  [OK] 56 servicios de aplicación que orquestan los casos de uso
  [OK] 41 opciones de menú PowerShell operacionales
  [OK] 8/8 requisitos de verificación de infraestructura cumplidos

El sistema demuestra conformidad estructural con 11 estándares
internacionales ISO/IEC, garantizando arquitectura escalable,
segura por diseño y auditablemente mantenible.

No Conformidades Identificadas: 5 (NC-01 a NC-05)
Clasificación: 1 Crítica, 2 Altas, 2 Medias
Resolución requerida antes de despliegue en producción.

Fecha de Auditoría: 08 de marzo de 2026
Auditor: Bryan Stiven Bonilla Rosas
Cargo: Arquitecto de Software | Auditor de Sistemas | Desarrollador Full Stack
Email: elviejobray@gmail.com | GitHub: @ELVIEJOBRAY

Firma: ___________________________________
═══════════════════════════════════════════════════════════════════════════
```

— FIN DEL DOCUMENTO —

SGDD v1.0.0 · SRS-DIA-001 · Documento Técnico de Ingeniería y Auditoría

Bryan Stiven Bonilla Rosas · @ELVIEJOBRAY · 08/03/2026